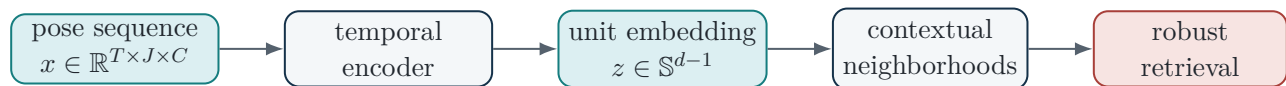

Contextual Similarity Optimization for Robust Pose-Sequence Retrieval

A beginner-first mathematical and implementation guide to
Liao, Tsiligkaridis, and Kulis (ICML 2023)



Audience. Begins below the level of the paper: variables, indices, indicators, vectors, matrices, norms, dot products, and losses are introduced explicitly. It then develops the paper's equations from first principles, derives the neighborhood objective, explains the non-standard gradients, and maps the method into a controlled pose-retrieval experiment.

Expanded beginner-first study edition
July 2026

How to use this guide today

Two routes through the book

Foundation route. Read chapters 1 to 4 in order. This route begins with the meaning of indices, indicators, vectors, matrices, and losses before using the paper’s notation.

Rapid discussion route. Read the research claim below, then figure 6.1, the five central equations (4.3) and (P2) to (P6), the ranking proposition in section 11.3, the implementation plan in chapter 15, and the discussion script in chapter 17.

The one-sentence research claim

Test whether a neighborhood-aware metric-learning objective preserves the ranking quality of pose-sequence embeddings better than standard objectives when query skeletons are jittered, incomplete, temporally damaged, or produced by a shifted pose estimator.

The contextual paper demonstrates robustness to *label noise and overfitting in image retrieval*. It does **not** establish robustness to corrupted pose coordinates. The proposed project is compelling precisely because that transfer is an open, falsifiable hypothesis rather than a guaranteed consequence.

Equations marked **P1–P31** reproduce or algebraically restate the numbered equations in Liao, Tsiligkaridis, and Kulis. Motion-specific equations, implementation recommendations, and robustness metrics are explicitly labeled as adaptations or proposed extensions. One line in the paper’s printed Algorithm 1 appears inconsistent with its Eq. (5): the algorithm line shows an MSE between contextual and cosine similarity, while Eq. (5), the proof, and the released code optimize contextual similarity against the binary label-similarity matrix. This guide follows Eq. (5) and the reference implementation.

How every difficult equation should be studied

For each major equation, first identify the output object, then read every symbol aloud, then compute a tiny numerical example, and only then study the formal or implementation-level interpretation. The book intentionally repeats core ideas at increasing levels of maturity.

Contents

How to use this guide today	i
1 How to Read the Mathematics in This Guide	1
1.1 The explanation ladder used throughout the book	1
1.2 Scalars, vectors, matrices, sets, and functions	1
1.2.1 A scalar is one number	2
1.2.2 A vector is an ordered list of numbers	2
1.2.3 A matrix is a table of numbers	2
1.2.4 A set is a collection of distinct objects	2
1.2.5 A function is a rule	2
1.3 Indices: what do i , j , and p mean?	3
1.4 A complete explanation of the label-similarity formula	3
1.4.1 Pass 1: the simplest possible meaning	3
1.4.2 Pass 2: read the expression aloud	3
1.4.3 Pass 3: symbol-by-symbol dictionary	4
1.4.4 Pass 4: the indicator function by itself	4
1.4.5 Pass 5: a four-sample example	4
1.4.6 Pass 6: the mature interpretation	5
1.5 Common operators used in the paper	5
1.5.1 Equality, definition, and membership	5
1.5.2 Summation notation	5
1.5.3 Conditions underneath a summation	6
1.5.4 Transpose	6
1.5.5 Norm	6
1.5.6 Dot product	6
1.5.7 The positive-part operator	6
1.5.8 Set intersection and complement	7
1.6 How a matrix formula becomes code	7
2 From a Pose Sequence to a Point in Embedding Space	8
2.1 Start with the physical data	8
2.2 Why the raw tensor is not directly used for retrieval	8
2.3 What an embedding coordinate means	9
2.4 Normalization with actual numbers	9
2.5 The mature representation-learning view	9
3 How a Loss Function Trains a Neural Network	10
3.1 The training loop in its simplest form	10
3.2 What a loss function is	10
3.3 Why use the square?	11
3.4 A derivative is a local sensitivity	11
3.5 Gradient descent	11
3.6 The chain rule in the current project	11

3.7	Why discrete operations cause a problem	12
4	Retrieval Geometry from First Principles	13
4.1	Retrieval is a ranking problem	13
4.1.1	Start with the practical task	13
4.1.2	Formal notation for the gallery	13
4.1.3	Relevant and irrelevant items	14
4.2	Similarity and distance are two ways to order points	14
4.2.1	Similarity: larger means closer	14
4.2.2	Distance: smaller means closer	14
4.3	Embeddings and normalization	14
4.3.1	The raw network output	14
4.3.2	Read the normalization formula symbol by symbol	15
4.3.3	Why normalization is important	15
4.4	The dot product and cosine similarity	15
4.4.1	Read the formula aloud	15
4.4.2	Compute a dot product by hand	15
4.4.3	Interpret the range $[-1, 1]$	16
4.5	Why squared Euclidean distance becomes $2 - 2s_{ij}$	16
4.5.1	Pass 1: the result in plain language	16
4.5.2	Pass 2: explain the left-hand side	16
4.5.3	Pass 3: derive every algebraic step	16
4.5.4	Pass 4: check the endpoints	17
4.5.5	Pass 5: why the rankings are identical	17
4.6	The label-similarity matrix	17
4.6.1	From one pair to a full matrix	18
4.6.2	The learned matrix and the target matrix	18
5	Metric-Learning Baselines and Balanced Batches	20
5.1	Begin with the purpose of a mini-batch	20
5.2	The two letters P and K	20
5.3	Why batch composition is part of the method	20
5.4	Pairwise contrastive loss	21
5.5	Triplet and supervised contrastive baselines	21
6	The Full Contextual-Similarity Pipeline	23
6.1	One picture before the details	23
6.2	Dimensions and tensor meanings	23
7	Step 1: Differentiable Neighborhoods	24
7.1	Start with a physical picture of a neighborhood	24
7.2	The boundary distance	24
7.3	The Heaviside operation and the heuristic gradient	25
7.4	The $k + \epsilon$ neighborhood	26
7.4.1	Read Equation P2 from the inside outward	26
7.4.2	Why the minus signs produce a distance comparison	26
7.4.3	Numerical example	26
7.5	Why stop the gradient through the boundary?	27

7.6	Why not use a sigmoid?	28
8	Step 2: Shared Neighborhood Context	29
8.1	Start with a social-structure question	29
8.2	Turn sets into binary rows	29
8.3	Neighbor overlap as a dot product	30
8.4	Why count shared non-neighbors?	30
8.5	The intermediate contextual score W_1	31
8.5.1	Read Equation P3 one layer at a time	31
8.5.2	A complete numerical calculation	31
8.6	Algebra when $\epsilon = 0$	32
9	Step 3: Reciprocal Query Expansion and Symmetry	33
9.1	Start with two refinements	33
9.2	Why nearest-neighbor relations are directional	33
9.3	Query expansion as local evidence averaging	33
9.4	Reciprocal close neighbors	34
9.5	Query expansion	34
9.5.1	Symbol-by-symbol interpretation of reciprocal query expansion	34
10	The Complete Objective and the Role of Every Term	36
10.1	Begin with the idea of matching two matrices	36
10.2	Why the final objective contains three terms	36
10.3	Contextual loss	37
10.3.1	Read the contextual loss as four nested operations	37
10.4	Similarity regularizer	38
10.5	The full objective	38
10.5.1	How the weighting coefficients work	38
10.6	The decomposability gap	38
10.7	Hyperparameter map	39
11	Why Zero Contextual Loss Means Correct Ranking	40
11.1	How to read a mathematical proposition	40
11.2	What “correct ranking” means formally	40
11.3	The paper’s proposition	40
11.3.1	Translate every quantifier in the ranking condition	41
11.4	Forward direction: correct ranking implies zero loss	41
11.5	Backward direction: zero loss implies correct ranking	41
11.6	What the assumptions are doing	42
12	Gradient Intuition and Semantic Consistency	43
12.1	Begin with the difference between a forward value and a backward signal	43
12.2	What a pairwise gradient means geometrically	43
12.3	Simplified gradient view	43
12.4	Why apparently “wrong” pairwise gradients can generalize	44
12.5	Stepwise gradient interpretation	44

13 A Worked Numerical Example	45
13.1 Setup	45
13.2 A true positive pair: (1, 2)	45
13.3 A wrong negative pair: (1, 5)	45
13.4 A missing positive pair: (1, 4)	46
13.5 What query expansion would do	46
14 Adapting the Mathematics to Pose Sequences	47
14.1 The input tensor	47
14.2 Pose normalization	47
14.3 Controlled corruption operators	47
14.3.1 Gaussian joint jitter	47
14.3.2 Joint or limb dropout	48
14.3.3 Frame dropout	48
14.3.4 View rotation in 3D	48
14.4 The central hypothesis in mathematical form	48
15 Implementation Blueprint	49
15.1 Recommended staged implementation	49
15.2 Reference PyTorch implementation	49
15.3 Training loop for pose sequences	52
15.4 Sampler invariant	52
15.5 Complexity and memory	52
15.6 Unit tests before training	53
16 Retrieval Evaluation and a Fair Experimental Design	54
16.1 Begin with one ranked list	54
16.2 Query/gallery construction	54
16.3 Recall at K	54
16.4 Average precision and mAP	55
16.5 Robustness curves	55
16.6 Core comparison matrix	56
16.7 Fairness controls	56
16.8 Recommended experiment order	56
17 Discussion Preparation	58
17.1 A 60-second explanation	58
17.2 The five equations to narrate	58
17.3 Likely questions and precise answers	58
17.4 Three decisions to ask the professor	59
17.5 Claims to avoid	59
17.6 Final mental model	60
A Notation Reference	61

B Detailed Algebra and Gradient Notes	62
B.1 Complement-intersection simplification	62
B.2 Step-1 surrogate gradient	62
B.3 Step-2 context gradient	62
B.4 Logical AND as multiplication	63
C Implementation Checklist	64
D Two-Minute Detachable Cheat Sheet	65
References	66

List of Figures

1.1	The label-similarity value is a numerical encoding of a yes/no statement.	3
2.1	A pose sequence is a three-axis array: time, joint identity, and coordinate.	8
2.2	The retrieval model converts a variable, structured motion sequence into one fixed-length point.	9
3.1	Training repeatedly measures an error and changes the network parameters to reduce it. .	10
4.1	A retrieval model predicts an ordering of database items.	13
4.2	Unit-normalized embeddings turn similarity into angular geometry.	16
4.3	With samples grouped by class, the target similarity matrix has visible blocks.	18
4.4	The main contribution is the intermediate transformation from pairwise geometry to contextual geometry.	18
5.1	A balanced batch is a structural assumption, not a minor dataloader choice.	21
5.2	Standard losses reason through pairs or triplets; contextual loss also reasons through the local graph around each sample.	22
6.1	The complete training computation. Every contextual quantity is ultimately a function of the cosine-similarity matrix.	23
7.1	A neighborhood is a binary partition induced by a distance threshold.	24
7.2	The k -th neighbor establishes the threshold; ϵ adds a small margin.	24
7.3	The method uses an exact discrete neighborhood in the forward pass and a straight-through-style heuristic in the backward pass.	25
7.4	ϵ expands the boundary beyond the current k -th neighbor and acts as a ranking margin. .	27
7.5	A sharper sigmoid is a better forward approximation but has useful gradients over a narrower region.	28
8.1	Context compares local structures, not only the direct line between two points.	29
8.2	Binary neighborhood overlap is an ordinary dot product.	30
8.3	M_+ measures shared inclusion; M_- measures shared exclusion.	30
9.1	Query expansion replaces one point's estimate with a local consensus estimate.	34
9.2	Reciprocity filters asymmetric nearest-neighbor accidents.	34
10.1	The three terms solve different failure modes.	38
10.2	Why the hybrid objective is stronger than contextual loss alone.	39
11.1	At the optimum, the neighbor graph has exactly the same block structure as label agreement.	42
12.1	A contextual error propagates through a neighborhood.	44
13.1	The numerical example as a neighborhood-repair problem.	46

14.1	The loss does not care whether z came from an image or a motion encoder. Only the encoder-side computation changes.	47
14.2	Schematic pose corruptions. These are controlled test interventions, not claims that one synthetic operator fully represents a real pose estimator.	48
15.1	A low-risk sequence that isolates bugs and scientific effects.	49
16.1	Schematic robustness curves only—not experimental results. The target result is a smaller degradation slope, not necessarily the highest clean point.	55

How to Read the Mathematics in This Guide

1.1 The explanation ladder used throughout the book

This edition deliberately explains each important idea in several passes. The passes are not repetitions. Each one adds a new level of precision.

1. **Start with the task.** What is the computation trying to accomplish?
2. **Use a tiny concrete example.** Work with two or four samples rather than an abstract dataset.
3. **Read the formula aloud.** Translate every symbol into a sentence.
4. **Compute it by hand.** Substitute actual numbers.
5. **State the formal interpretation.** Explain why the formula has the required mathematical properties.
6. **Connect it to code.** Identify tensor shapes and operations.

The rule for reading a formula

Do not begin by trying to understand the entire expression at once. First identify:

what is being defined what information enters it what type of object comes out.

Only after those three questions are answered should you inspect the individual operators.

1.2 Scalars, vectors, matrices, sets, and functions

Most confusion in machine-learning mathematics comes from not knowing what kind of object a symbol represents. The same-looking letter can refer to a single number, a list of numbers, or a two-dimensional table. These objects support different operations.

Object	Example	Meaning in this project
Scalar	$s_{ij} \in \mathbb{R}$	one similarity value between two samples
Vector	$z_i \in \mathbb{R}^d$	one embedding containing d numbers
Matrix	$S \in \mathbb{R}^{n \times n}$	all pairwise similarities in a batch
Set	$N_k(i)$	the collection of the k nearest samples to i
Function	$f_\theta(x)$	a rule, implemented by a neural network, that maps an input to an embedding
Binary value	$y_{ij} \in \{0, 1\}$	an answer to a yes/no question

1.2.1 A scalar is one number

A scalar has no list structure. Examples are

$$3.2, \quad -1, \quad 0.87.$$

In the contextual paper, s_{ij} is a scalar. It is one number telling us how similar sample i is to sample j .

1.2.2 A vector is an ordered list of numbers

A vector with three coordinates can be written as

$$z_i = \begin{bmatrix} 0.6 \\ 0.8 \\ 0 \end{bmatrix} \in \mathbb{R}^3.$$

The superscript 3 in $\mathbb{R}^3$ means that the vector has three real-valued coordinates. In the actual project, the dimension may be 128, 256, or another chosen value.

Why order matters

The vectors $[1, 0]^\top$ and $[0, 1]^\top$ contain the same two numbers, but they are not the same vector. Coordinate 1 and coordinate 2 represent different learned directions in the embedding space.

1.2.3 A matrix is a table of numbers

A matrix has rows and columns. For three samples, a similarity matrix may look like

$$S = \begin{bmatrix} 1 & 0.8 & -0.2 \\ 0.8 & 1 & 0.1 \\ -0.2 & 0.1 & 1 \end{bmatrix}.$$

The entry in row i and column j is written S_{ij} or s_{ij} . The diagonal entries are all 1 because every normalized vector is perfectly similar to itself.

1.2.4 A set is a collection of distinct objects

The notation

$$N_3(i) = \{i, 2, 7\}$$

means that the three nearest items to sample i are sample i itself, sample 2, and sample 7. Curly braces describe membership, not an ordered vector. In the paper, sets are often converted to binary indicator vectors so that set operations can be implemented with matrix multiplication.

1.2.5 A function is a rule

The expression

$$z_i = f_\theta(x_i)$$

means: apply the function f_θ to input x_i and call the result z_i . The subscript θ represents all trainable parameters of the neural network. During gradient descent, those parameters change.

1.3 Indices: what do i , j , and p mean?

An index is an integer used to identify an item. If a mini-batch contains n samples, then

$$i, j, p \in \{1, 2, \dots, n\}.$$

The symbols i and j are not values of the input. They are addresses. They tell us which samples are being discussed.

Index	Sample	Label
1	first pose sequence	walk
2	second pose sequence	walk
3	third pose sequence	jump
4	fourth pose sequence	throw

Thus y_2 means the class label of sample 2, while y_{24} means a relation between samples 2 and 4. The extra index changes the meaning of the symbol.

1.4 A complete explanation of the label-similarity formula

The formula that often appears first in the paper is

$$y_{ij} = \mathbb{1}[y_i = y_j] \in \{0, 1\}. \quad (1.1)$$

We now unpack every part.

1.4.1 Pass 1: the simplest possible meaning

The formula asks one yes/no question:

Do sample i and sample j have the same class label?

If the answer is yes, assign the number 1. If the answer is no, assign the number 0.



Figure 1.1: The label-similarity value is a numerical encoding of a yes/no statement.

1.4.2 Pass 2: read the expression aloud

Read

$$y_{ij} = \mathbb{1}[y_i = y_j] \in \{0, 1\}$$

as follows:

“The pairwise label-similarity value for samples i and j equals the indicator of the statement that the label of sample i equals the label of sample j . The result must be either zero or one.”

1.4.3 Pass 3: symbol-by-symbol dictionary

Symbol	Meaning
i	index of the first sample
j	index of the second sample
y_i	class label attached to sample i
y_j	class label attached to sample j
$y_i = y_j$	a logical statement that is either true or false
$\mathbb{1}[\cdot]$	indicator function: convert true to 1 and false to 0
y_{ij}	the resulting pairwise binary similarity target
$\in$	“is an element of” or “belongs to”
$\{0, 1\}$	the set containing exactly the values zero and one

1.4.4 Pass 4: the indicator function by itself

For any logical statement A , define

$$\mathbb{1}[A] = \begin{cases} 1, & \text{if } A \text{ is true,} \\ 0, & \text{if } A \text{ is false.} \end{cases}$$

For example,

$$\mathbb{1}[3 < 5] = 1, \quad \mathbb{1}[8 = 2] = 0.$$

The indicator function lets us place logical information inside arithmetic expressions. Once a statement has been converted to 0 or 1, we can add it, multiply it, average it, or put it into a matrix.

1.4.5 Pass 5: a four-sample example

Suppose the label vector is

$$\mathbf{y} = \begin{bmatrix} \text{walk} \\ \text{walk} \\ \text{jump} \\ \text{throw} \end{bmatrix}.$$

Then

$$y_{12} = \mathbb{1}[\text{walk} = \text{walk}] = 1,$$

while

$$y_{13} = \mathbb{1}[\text{walk} = \text{jump}] = 0.$$

Applying the rule to every ordered pair gives

$$Y = \begin{bmatrix} 1 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

Why the matrix is symmetric

If sample i has the same label as sample j , then sample j also has the same label as sample i . Therefore

$$y_{ij} = y_{ji},$$

so $Y = Y^\top$.

Why the diagonal is one

Every label equals itself. Therefore $y_i = y_i$ is always true and

$$y_{ii} = 1.$$

The contextual loss later removes diagonal terms because self-similarity is trivial and does not teach the model how to rank different samples.

1.4.6 Pass 6: the mature interpretation

The label-similarity matrix Y is the adjacency matrix of a graph. Each sample is a node. Two nodes are connected when they share a class label. Training attempts to make the learned similarity structure resemble this target graph. Contextual loss differs from a purely pairwise loss because it compares not only individual edges but also the local graph surrounding each node.

1.5 Common operators used in the paper**1.5.1 Equality, definition, and membership**

The symbol $=$ may report an equality or define a new object. Context usually tells which role it has. The symbol $\in$ reports membership:

$$z_i \in \mathbb{R}^d$$

means that z_i is a d -dimensional real vector.

1.5.2 Summation notation

The expression

$$\sum_{p=1}^n a_p$$

means

$$a_1 + a_2 + \cdots + a_n.$$

The letter p is a temporary index. It takes every integer value from 1 through n .

For example,

$$\sum_{p=1}^4 a_p = a_1 + a_2 + a_3 + a_4.$$

1.5.3 Conditions underneath a summation

A sum such as

$$\sum_{i,j | i \neq j} (y_{ij} - w_{ij})^2$$

means: include all pairs (i, j) except pairs for which $i = j$. The vertical bar can be read as “such that.”

1.5.4 Transpose

For a column vector

$$z = \begin{bmatrix} z_1 \\ z_2 \\ z_3 \end{bmatrix}, \quad z^T = \begin{bmatrix} z_1 & z_2 & z_3 \end{bmatrix}.$$

The transpose turns rows into columns and columns into rows. It enables matrix expressions such as ZZ^T .

1.5.5 Norm

The Euclidean or L_2 norm is the length of a vector:

$$\|z\|_2 = \sqrt{z_1^2 + z_2^2 + \cdots + z_d^2}.$$

For $z = [3, 4]^T$,

$$\|z\|_2 = \sqrt{3^2 + 4^2} = 5.$$

1.5.6 Dot product

For two vectors $a, b \in \mathbb{R}^d$,

$$a^T b = \sum_{r=1}^d a_r b_r.$$

This means: multiply matching coordinates, then add the products. For

$$a = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}, \quad b = \begin{bmatrix} 4 \\ 0 \\ -1 \end{bmatrix},$$

we get

$$a^T b = (1)(4) + (2)(0) + (3)(-1) = 1.$$

1.5.7 The positive-part operator

The notation

$$[u]_+ = \max(u, 0)$$

means that negative values are replaced by zero. It is the scalar ReLU operation. For example,

$$[-2]_+ = 0, \quad [0.7]_+ = 0.7.$$

Metric-learning margin losses use this operation so that already-correct pairs contribute no loss.

1.5.8 Set intersection and complement

If

$$A = \{1, 2, 4\}, \quad B = \{2, 4, 5\},$$

then their intersection is

$$A \cap B = \{2, 4\}.$$

The intersection contains elements that appear in both sets. Contextual similarity relies heavily on the size of neighborhood intersections.

1.6 How a matrix formula becomes code

Suppose $Z \in \mathbb{R}^{n \times d}$ contains one embedding per row. Then

$$S = ZZ^\top$$

produces an $n \times n$ matrix. Entry (i, j) is

$$S_{ij} = z_i^\top z_j.$$

In PyTorch, this is simply

```
1 S = Z @ Z.T
```

The symbol `@` means matrix multiplication. This single operation computes all n^2 pairwise dot products in parallel.

Checkpoint before continuing

You are ready for the next chapter when you can explain the difference between y_i and y_{ij} , state what an indicator function does, and identify whether z_i , S , and $N_k(i)$ are respectively a vector, matrix, or set.

From a Pose Sequence to a Point in Embedding Space

2.1 Start with the physical data

A pose sequence is a series of estimated body configurations. At each time step, a pose estimator returns the coordinates of several body joints. A convenient mathematical representation is

$$x_i \in \mathbb{R}^{T \times J \times C},$$

where:

- i identifies the motion clip;
- T is the number of time frames;
- J is the number of body joints;
- C is the number of coordinates per joint, usually 2 or 3.

If a clip contains $T = 120$ frames, $J = 25$ joints, and three coordinates (x, y, z) per joint, then the clip contains

$$120 \times 25 \times 3 = 9000$$

coordinate values before any extra confidence values or features are added.

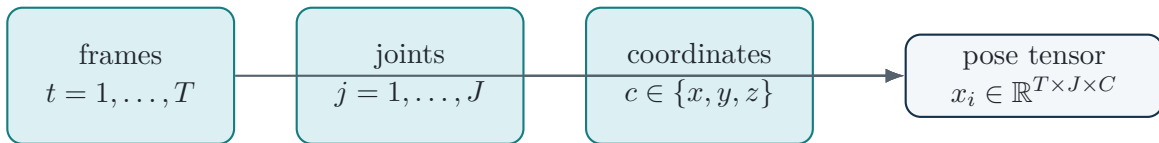


Figure 2.1: A pose sequence is a three-axis array: time, joint identity, and coordinate.

2.2 Why the raw tensor is not directly used for retrieval

Two recordings of the same action can differ in length, body proportions, camera viewpoint, execution speed, and pose-estimation error. Directly subtracting all raw coordinates would therefore measure many nuisance differences. We instead learn an encoder

$$h_i = f_\theta(x_i) \in \mathbb{R}^d.$$

The encoder compresses the entire sequence into a fixed-length vector. The training objective determines what information that vector should preserve.

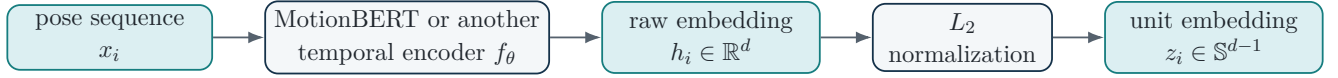


Figure 2.2: The retrieval model converts a variable, structured motion sequence into one fixed-length point.

2.3 What an embedding coordinate means

A learned coordinate usually does not have a direct human-readable meaning. Coordinate 17 is not necessarily “elbow angle” and coordinate 44 is not necessarily “speed.” The network distributes useful information across many coordinates. What matters is the geometry of the complete vector: samples that should retrieve one another must point in similar directions.

2.4 Normalization with actual numbers

Suppose an encoder outputs

$$h_i = \begin{bmatrix} 3 \\ 4 \end{bmatrix}.$$

Its length is

$$\|h_i\|_2 = \sqrt{3^2 + 4^2} = 5.$$

The normalized embedding is

$$z_i = \frac{h_i}{\|h_i\|_2} = \begin{bmatrix} 3/5 \\ 4/5 \end{bmatrix} = \begin{bmatrix} 0.6 \\ 0.8 \end{bmatrix}.$$

Checking the new length,

$$\|z_i\|_2 = \sqrt{0.6^2 + 0.8^2} = 1.$$

What normalization changes and preserves

Normalization changes vector magnitude but preserves vector direction. After normalization, the network cannot make two samples appear similar merely by increasing both vector lengths. Similarity depends on angle.

2.5 The mature representation-learning view

The model learns a map

$$f_\theta : \mathcal{X} \rightarrow \mathbb{S}^{d-1},$$

from the space $\mathcal{X}$ of pose sequences to the unit sphere $\mathbb{S}^{d-1}$. Metric learning does not prescribe the coordinates of each point individually. It constrains relationships among points. The contextual paper strengthens this relational view by constraining local neighborhoods rather than only isolated pairs.

How a Loss Function Trains a Neural Network

3.1 The training loop in its simplest form

A neural network begins with parameters θ that do not yet produce useful retrieval geometry. Training repeats four operations:

1. compute embeddings using the current parameters;
2. measure how wrong the embeddings are using a loss function;
3. compute how the loss changes when each parameter changes;
4. adjust the parameters in a direction that reduces the loss.

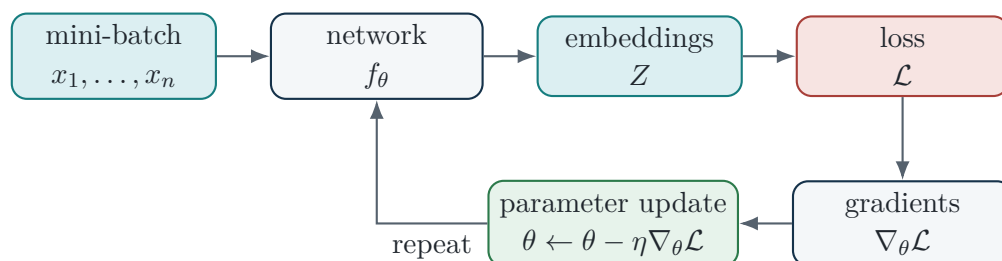


Figure 3.1: Training repeatedly measures an error and changes the network parameters to reduce it.

3.2 What a loss function is

A loss function returns one scalar. Smaller is better. A basic squared-error loss is

$$\ell = (t - p)^2,$$

where t is a target and p is a prediction.

If $t = 1$ and $p = 0.8$, then

$$\ell = (1 - 0.8)^2 = 0.04.$$

If $p = 0.2$, then

$$\ell = (1 - 0.2)^2 = 0.64.$$

The second prediction is farther from the target and receives a larger penalty.

3.3 Why use the square?

The square has three useful effects:

- negative and positive errors both become nonnegative;
- large errors receive disproportionately larger penalties;
- the function is smooth and easy to differentiate.

3.4 A derivative is a local sensitivity

For a scalar function $\ell(p)$, the derivative

$$\frac{d\ell}{dp}$$

answers: if p changes by a very small amount, how quickly does the loss change?

For

$$\ell(p) = (t - p)^2,$$

the derivative is

$$\frac{d\ell}{dp} = 2(p - t).$$

If $t = 1$ and $p = 0.2$, then

$$\frac{d\ell}{dp} = 2(0.2 - 1) = -1.6.$$

The negative sign says that increasing p will decrease the loss.

3.5 Gradient descent

For many parameters, collect all partial derivatives into the gradient

$$\nabla_{\theta}\mathcal{L}.$$

The basic update is

$$\theta \leftarrow \theta - \eta \nabla_{\theta}\mathcal{L},$$

where $\eta > 0$ is the learning rate. Subtracting the gradient moves parameters locally downhill.

The role of the loss

The loss does not directly move embeddings. It creates gradients. Those gradients alter network parameters. On the next forward pass, the altered network produces different embeddings.

3.6 The chain rule in the current project

The contextual loss depends on contextual similarity, which depends on neighborhoods, which depend on distances, which depend on cosine similarities, which depend on embeddings, which depend on network parameters:

$$\theta \longrightarrow Z \longrightarrow S \longrightarrow D \longrightarrow N \longrightarrow W \longrightarrow \mathcal{L}.$$

The chain rule propagates sensitivity backward through this sequence. Schematically,

$$\frac{\partial \mathcal{L}}{\partial \theta} = \frac{\partial \mathcal{L}}{\partial W} \frac{\partial W}{\partial N} \frac{\partial N}{\partial D} \frac{\partial D}{\partial S} \frac{\partial S}{\partial Z} \frac{\partial Z}{\partial \theta}.$$

This expression is not ordinary scalar multiplication in implementation; it summarizes a sequence of Jacobian-vector products performed by automatic differentiation.

3.7 Why discrete operations cause a problem

A hard nearest-neighbor membership decision changes suddenly from 0 to 1. Such a decision is constant almost everywhere. Its true derivative is zero almost everywhere, so ordinary backpropagation cannot tell the model how to change a distance to cross the membership boundary.

The contextual paper keeps the exact binary decision in the forward pass but defines a heuristic nonzero derivative in the backward pass. This is the key optimization device studied in chapter 7.

Checkpoint

The mature point to remember is that contextual similarity is not difficult because its forward calculation is impossible. It is difficult because the calculation contains discrete neighborhood decisions that block ordinary gradients.

Retrieval Geometry from First Principles

4.1 Retrieval is a ranking problem

4.1.1 Start with the practical task

Assume that a database contains many pose sequences. A new query sequence arrives. The system should return the database sequences in an order such that the most relevant examples appear first.

This output is different from classification. A classifier returns one label, such as “throw.” A retrieval system returns an ordered list of examples:

$$\text{query} \longrightarrow [\text{extmostsimilar}, \text{second most similar}, \dots].$$

The ordering itself is the prediction.

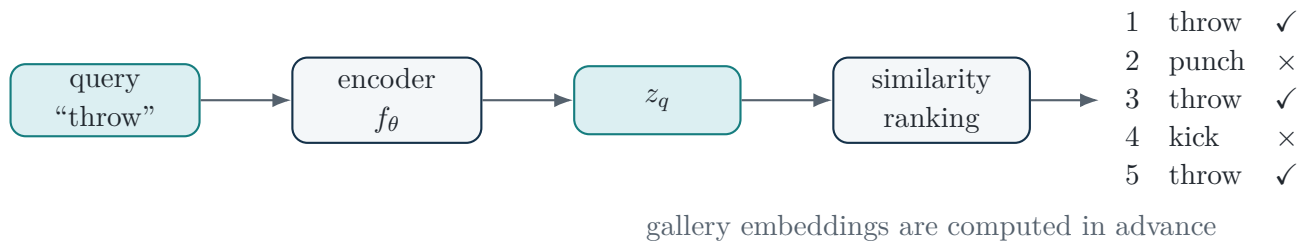


Figure 4.1: A retrieval model predicts an ordering of database items.

4.1.2 Formal notation for the gallery

Let the query be q . Let the gallery be the set

$$\mathcal{G} = \{g_1, g_2, \dots, g_m\}.$$

Here m is the number of gallery items. The retrieval system assigns one score to each pair (q, g_j) and sorts the items by score.

An ordering can be represented by a permutation π . The ranked output is

$$g_{\pi(1)}, g_{\pi(2)}, \dots, g_{\pi(m)}.$$

The function π simply records which original gallery index occupies each rank. For example, if the original items g_4, g_2, g_1, g_3 are returned in that order, then

$$\pi(1) = 4, \quad \pi(2) = 2, \quad \pi(3) = 1, \quad \pi(4) = 3.$$

4.1.3 Relevant and irrelevant items

For the first NTU120 experiment, two sequences can be declared relevant when they share the same action class. Define

$$\text{rel}(q, g_j) = \mathbb{1}[y_q = y_j].$$

This is the same indicator construction introduced in section 1.4.

What this relevance definition does not claim

Class-level relevance means that two clips belong to the same action category. It does not mean that they have identical biomechanics, timing, quality, or technique. NTU120 provides an objective first benchmark, not a final definition of fine-grained motion similarity.

4.2 Similarity and distance are two ways to order points

4.2.1 Similarity: larger means closer

A similarity score is designed so that larger values indicate a stronger match. If

$$s(q, g_1) = 0.92, \quad s(q, g_2) = 0.31,$$

then g_1 should rank ahead of g_2 .

4.2.2 Distance: smaller means closer

A distance is designed so that smaller values indicate a stronger match. If

$$D(q, g_1) = 0.16, \quad D(q, g_2) = 1.38,$$

then again g_1 should rank ahead of g_2 .

The paper uses both languages. It uses cosine similarity to describe retrieval and squared Euclidean distance to define neighborhood thresholds. Because embeddings are normalized, the two produce exactly the same ranking.

4.3 Embeddings and normalization

4.3.1 The raw network output

Let

$$h_i = f_\theta(x_i) \in \mathbb{R}^d.$$

Read this as:

“The neural network with parameters θ processes input sample x_i and returns a d -dimensional vector called h_i .”

The vector h_i may have any nonzero length. The paper normalizes it:

$$z_i = \frac{h_i}{\|h_i\|_2}, \quad \|z_i\|_2 = 1. \quad (4.1)$$

4.3.2 Read the normalization formula symbol by symbol

Part	Meaning
h_i	raw embedding of sample i
$\ h_i\ _2$	Euclidean length of that raw embedding
$h_i / \ h_i\ _2$	divide every coordinate by the same positive length
z_i	resulting normalized embedding
$\ z_i\ _2 = 1$	the normalized embedding lies on the unit sphere

4.3.3 Why normalization is important

Without normalization, a dot product is affected by both direction and magnitude:

$$h_i^\top h_j = \|h_i\|_2 \|h_j\|_2 \cos \phi.$$

The angle ϕ captures directional alignment, but the two lengths also multiply the score. After normalization, both lengths equal one:

$$z_i^\top z_j = \cos \phi.$$

This makes the score easier to interpret and keeps it inside a fixed interval.

4.4 The dot product and cosine similarity

For normalized embeddings, the paper defines

$$s_{ij} = \langle z_i, z_j \rangle = z_i^\top z_j \in [-1, 1]. \quad (4.2)$$

4.4.1 Read the formula aloud

“The cosine similarity between samples i and j equals the dot product of their normalized embedding vectors. It must lie between negative one and positive one.”

4.4.2 Compute a dot product by hand

Let

$$z_i = \begin{bmatrix} 0.6 \\ 0.8 \end{bmatrix}, \quad z_j = \begin{bmatrix} 0.8 \\ 0.6 \end{bmatrix}.$$

Then

$$\begin{aligned} s_{ij} &= z_i^\top z_j \\ &= (0.6)(0.8) + (0.8)(0.6) \\ &= 0.48 + 0.48 \\ &= 0.96. \end{aligned}$$

The value is close to one, so the vectors point in similar directions.

4.4.3 Interpret the range $[-1, 1]$

Value	Geometric meaning
$s_{ij} = 1$	same direction; maximum cosine similarity
$s_{ij} = 0$	perpendicular directions; no directional alignment
$s_{ij} = -1$	opposite directions; minimum cosine similarity

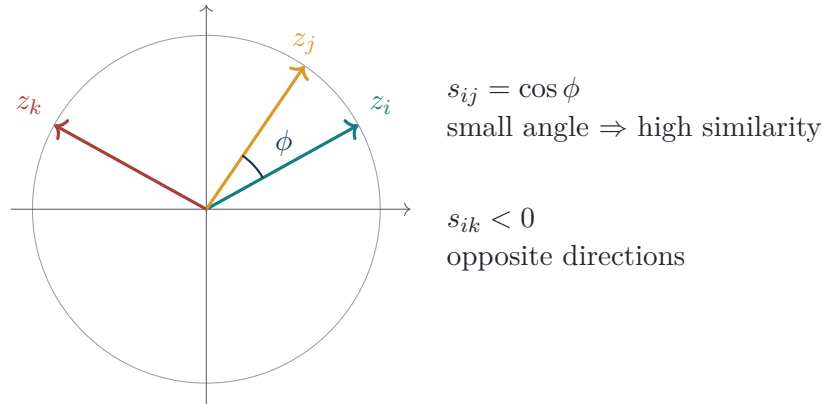


Figure 4.2: Unit-normalized embeddings turn similarity into angular geometry.

4.5 Why squared Euclidean distance becomes $2 - 2s_{ij}$

This identity is used repeatedly in the paper:

$$D(i, j) = \|z_i - z_j\|_2^2 = 2 - 2s_{ij}. \quad (4.3)$$

4.5.1 Pass 1: the result in plain language

For unit-length embeddings, knowing cosine similarity is enough to know squared Euclidean distance. A larger similarity always produces a smaller distance.

4.5.2 Pass 2: explain the left-hand side

The vector difference

$$z_i - z_j$$

points from one embedding to the other. Its norm is the straight-line distance between them. Squaring the norm removes the square root and is convenient for algebra.

4.5.3 Pass 3: derive every algebraic step

Start from the definition:

$$D(i, j) = \|z_i - z_j\|_2^2. \quad (4.4)$$

A squared norm can be written as a vector dotted with itself:

$$\|z_i - z_j\|_2^2 = (z_i - z_j)^\top (z_i - z_j). \quad (4.5)$$

Distribute the multiplication:

$$(z_i - z_j)^\top (z_i - z_j) = z_i^\top z_i - z_i^\top z_j - z_j^\top z_i + z_j^\top z_j. \quad (4.6)$$

The middle two scalar dot products are equal:

$$z_i^\top z_j = z_j^\top z_i = s_{ij}.$$

The first and last terms are squared lengths:

$$z_i^\top z_i = \|z_i\|_2^2 = 1, \quad z_j^\top z_j = \|z_j\|_2^2 = 1.$$

Substituting gives

$$\begin{aligned} D(i, j) &= 1 - s_{ij} - s_{ij} + 1 \\ &= 2 - 2s_{ij}. \end{aligned}$$

4.5.4 Pass 4: check the endpoints

If the vectors are identical, $s_{ij} = 1$:

$$D(i, j) = 2 - 2(1) = 0.$$

If the vectors are opposite, $s_{ij} = -1$:

$$D(i, j) = 2 - 2(-1) = 4.$$

Therefore $D(i, j) \in [0, 4]$.

4.5.5 Pass 5: why the rankings are identical

The function

$$D = 2 - 2s$$

is strictly decreasing in s . If $s_{i1} > s_{i2}$, then

$$2 - 2s_{i1} < 2 - 2s_{i2}.$$

Thus sorting by decreasing cosine similarity is identical to sorting by increasing squared Euclidean distance.

What changes and what does not

The numerical scale changes: similarity lies in $[-1, 1]$, while squared distance lies in $[0, 4]$. The order of neighbors does not change.

4.6 The label-similarity matrix

For a mini-batch of n labeled examples, define

$$y_{ij} = \mathbb{1}[y_i = y_j] \in \{0, 1\}. \quad (4.7)$$

This is the same formula studied in detail in section 1.4. It creates the target relation for every pair of samples.

4.6.1 From one pair to a full matrix

Collect all values into

$$Y = [y_{ij}] \in \{0, 1\}^{n \times n}.$$

Row i describes which batch samples are relevant to sample i . Column j describes which anchors regard sample j as relevant.

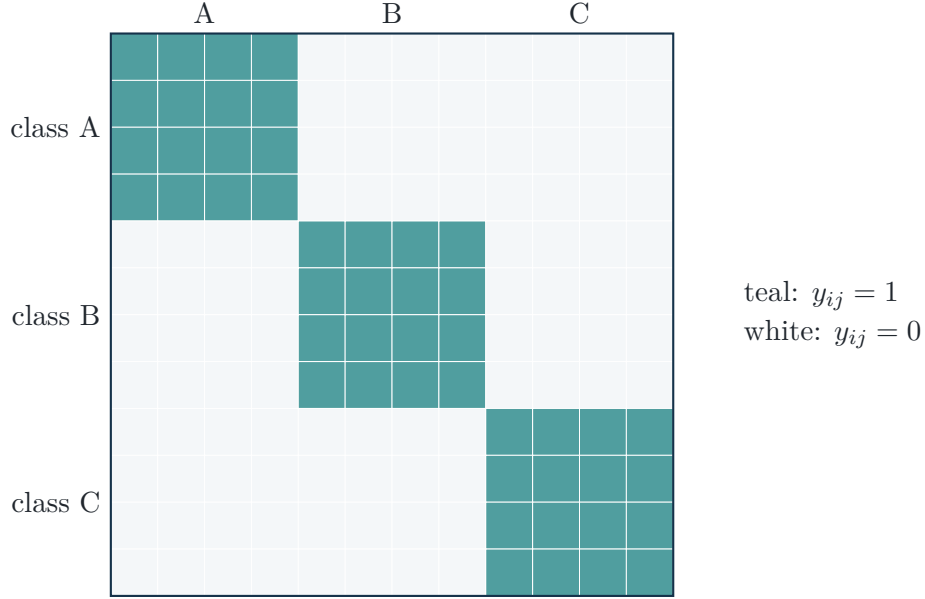


Figure 4.3: With samples grouped by class, the target similarity matrix has visible blocks.

4.6.2 The learned matrix and the target matrix

The learned cosine similarities form

$$S = [s_{ij}] \in [-1, 1]^{n \times n}.$$

The target relations form

$$Y = [y_{ij}] \in \{0, 1\}^{n \times n}.$$

A pairwise loss compares entries of S with pair labels. The contextual loss first transforms S into a new matrix W that describes neighborhood agreement, and then compares W with Y .



Figure 4.4: The main contribution is the intermediate transformation from pairwise geometry to contextual geometry.

What to say in the discussion

“Each sequence becomes a unit vector. The dot product is its cosine similarity. Because the vectors have unit length, squared Euclidean distance is exactly $2 - 2s_{ij}$, so similarity and distance produce the same neighbor ranking. The labels create a binary target matrix Y . The contextual method does not match S to Y directly; it first asks whether pairs occupy similar local neighborhoods.”

Metric-Learning Baselines and Balanced Batches

5.1 Begin with the purpose of a mini-batch

A dataset may contain tens of thousands of motion clips. The model usually does not process all of them at once. Instead, it processes a small collection called a mini-batch. All pairwise relationships used by the contextual loss are computed inside that mini-batch.

This makes batch composition mathematically important. If a batch accidentally contains only one example from each class, there are no same-class pairs to pull together. If one class dominates the batch, neighborhood sizes no longer line up cleanly with class size.

Plain-language definition of balanced sampling

Choose the same number of examples from several classes. For example, choose four action classes and four clips from each class. The batch then contains sixteen clips.

5.2 The two letters P and K

Many metric-learning implementations call this a $P \times K$ sampler:

- P : number of distinct classes selected for the batch;
- K : number of samples selected from each chosen class.

The total batch size is

$$n = PK.$$

The contextual paper uses k for the samples-per-class value, so this guide identifies $k = K$.

5.3 Why batch composition is part of the method

The paper uses a $P \times K$ sampler: choose P classes and K samples per class. To match the paper's notation, this guide writes

$$k = K, \quad n = Pk.$$

The contextual neighborhood size is **not a free top- k tuning parameter**: the paper sets it equal to the number of samples from each class in the mini-batch.

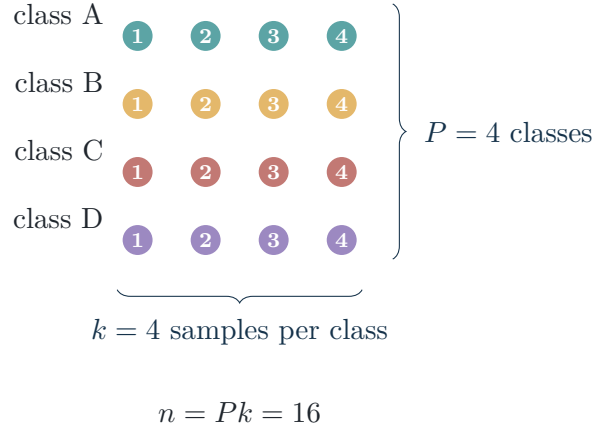


Figure 5.1: A balanced batch is a structural assumption, not a minor dataloader choice.

Why k equals samples per class

If ranking is perfect inside a balanced batch, each sample's k nearest items, counting itself, are exactly the k examples with the same label. This identity is what lets zero contextual loss coincide with correct within-batch ranking in the paper's proposition.

5.4 Pairwise contrastive loss

The contextual framework retains a standard pairwise term. In similarity form, the paper uses positive and negative margins δ_+ and δ_- :

$$\mathcal{L}_{\text{contrast}} = \frac{\sum_{i,j:y_{ij}=1} [\delta_+ - s_{ij}]_+}{|\{(i,j) : y_{ij} = 1, \delta_+ - s_{ij} > 0\}|} + \frac{\sum_{i,j:y_{ij}=0} [s_{ij} - \delta_-]_+}{|\{(i,j) : y_{ij} = 0, s_{ij} - \delta_- > 0\}|}. \quad (\text{P29})$$

Here $[a]_+ = \max(a, 0)$. A positive pair is penalized if its similarity is below δ_+ ; a negative pair is penalized if its similarity is above δ_- . Only margin-violating pairs contribute.

5.5 Triplet and supervised contrastive baselines

These are not part of the contextual paper's final formula, but they are critical controls for the proposed motion study.

For an anchor a , positive p , negative n , triplet loss is

$$\mathcal{L}_{\text{triplet}} = [D(a, p) - D(a, n) + m]_+. \quad (5.1)$$

It asks only for a relative inequality.

For supervised contrastive learning, define $P(i)$ as the set of same-label positives for anchor i and $A(i)$ as all other batch indices. With temperature τ ,

$$\mathcal{L}_{\text{supcon}} = \sum_i -\frac{1}{|P(i)|} \sum_{p \in P(i)} \log \frac{\exp(z_i^\top z_p / \tau)}{\sum_{a \in A(i)} \exp(z_i^\top z_a / \tau)}. \quad (5.2)$$

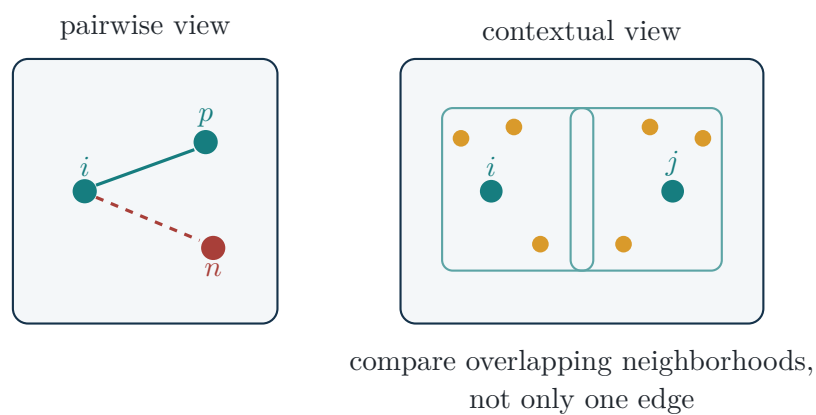


Figure 5.2: Standard losses reason through pairs or triplets; contextual loss also reasons through the local graph around each sample.

The Full Contextual-Similarity Pipeline

6.1 One picture before the details

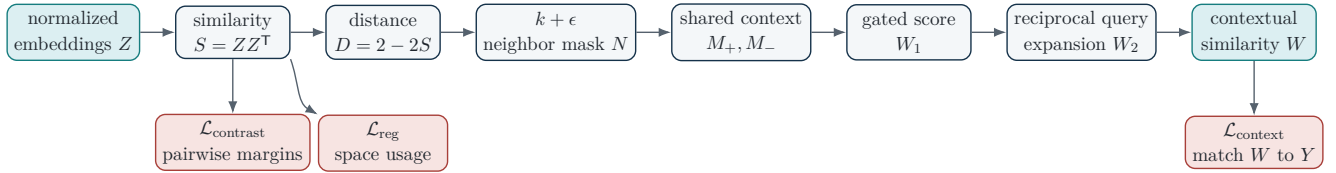


Figure 6.1: The complete training computation. Every contextual quantity is ultimately a function of the cosine-similarity matrix.

6.2 Dimensions and tensor meanings

Symbol	Shape	Meaning
Z	$n \times d$	one normalized embedding per mini-batch item
S	$n \times n$	cosine similarity, $S_{ij} = z_i^\top z_j$
D	$n \times n$	squared Euclidean distance, $D = 2\mathbf{1}\mathbf{1}^\top - 2S$ entrywise
Y	$n \times n$	binary label agreement
N	$n \times n$	differentiable-in-backward binary neighborhood indicator
M_+, M_-	$n \times n$	shared-neighbor and shared-non-neighbor counts
W_1, W_2, W	$n \times n$	progressively refined contextual similarities

The conceptual compression

The entire paper can be remembered as

embeddings $\rightarrow$ pairwise similarities $\rightarrow$ local graph $\rightarrow$ graph similarity $\rightarrow$ label graph.

The difficult part is making the discrete local graph trainable.

Step 1: Differentiable Neighborhoods

7.1 Start with a physical picture of a neighborhood

Place every embedding as a point in space. For one anchor point i , sort all batch points from nearest to farthest. The first k points, counting the anchor itself, form its k -nearest neighborhood.

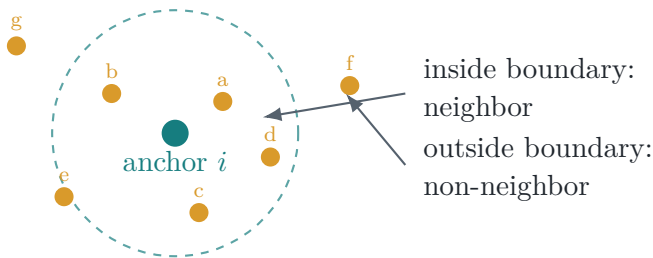


Figure 7.1: A neighborhood is a binary partition induced by a distance threshold.

The paper does not merely store the list of top- k indices. It expresses membership through a threshold test. That allows the forward pass to create a binary matrix and the backward pass to attach a custom gradient to the threshold.

7.2 The boundary distance

Let $p(i)$ denote the sample occupying rank k in the sorted distance list for anchor i . Then

$$D(i, p(i))$$

is the distance from anchor i to the boundary item. Any sample closer than this boundary is a neighbor. A small margin ϵ slightly expands the boundary.

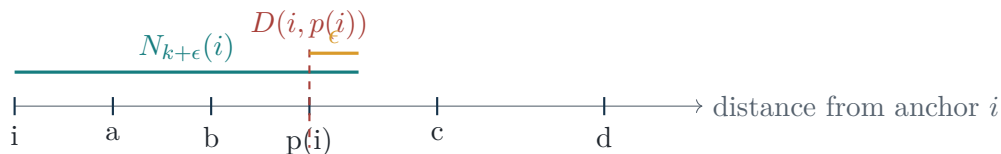


Figure 7.2: The k -th neighbor establishes the threshold; ϵ adds a small margin.

7.3 The Heaviside operation and the heuristic gradient

A top- k neighborhood is discrete. The paper expresses the threshold with a Heaviside function:

$$\text{Forward: } \theta(x) = \begin{cases} 1, & x \geq 0, \\ 0, & x < 0, \end{cases} \quad \text{Backward: } \frac{\partial \theta(x)}{\partial x} = \alpha, \quad (\text{P1})$$

where $\alpha > 0$ is a constant. The forward pass is exact and binary; the backward pass deliberately overrides the true derivative, which is zero almost everywhere and undefined at zero.

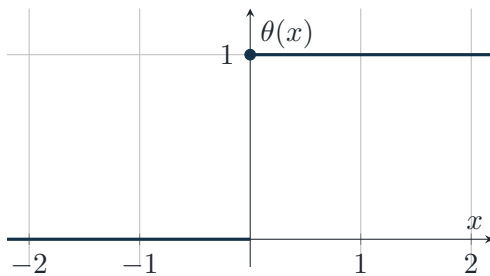
Read the Heaviside rule as a switch

The forward rule has one input number x and one binary output. When x is nonnegative, the switch is on and returns 1. When x is negative, the switch is off and returns 0. The symbol θ here is a function name; it is unrelated to the neural-network parameter collection also commonly denoted by θ in other contexts.

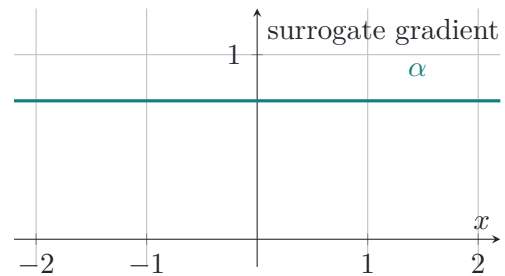
The true derivative of a step function is not useful for gradient descent. Away from the jump, a small change in x does not change the output, so the derivative is zero. At the jump, the derivative is not defined. The paper therefore replaces the true backward behavior with a chosen constant $\alpha > 0$.

Quantity	Interpretation
x	value being compared with zero
$\theta(x)$	exact binary decision used in the forward computation
$\partial\theta/\partial x$	sensitivity used during backpropagation
α	user-chosen positive surrogate sensitivity

The paper is not claiming that the mathematical derivative of the Heaviside function is a positive constant. It deliberately defines a different backward rule so that the discrete forward computation can still provide a learning signal.



(a) Exact binary forward pass.



(b) Constant backward signal.

Figure 7.3: The method uses an exact discrete neighborhood in the forward pass and a straight-through-style heuristic in the backward pass.

7.4 The $k + \epsilon$ neighborhood

Let $p(i)$ be the index of the k -th closest item to i , counting i itself. The paper defines

$$\mathbb{1}_{N_{k+\epsilon}}(i, j) = \theta(-D(i, j) + \text{sg}(D(i, p(i))) + \epsilon). \quad (\text{P2})$$

Equivalently,

$$j \in N_{k+\epsilon}(i) \iff D(i, j) \leq D(i, p(i)) + \epsilon.$$

7.4.1 Read Equation P2 from the inside outward

The expression

$$\mathbb{1}_{N_{k+\epsilon}}(i, j) = \theta(-D(i, j) + \text{sg}(D(i, p(i))) + \epsilon)$$

contains four conceptual steps.

1. Compute the candidate distance $D(i, j)$.
2. Compute the current boundary distance $D(i, p(i))$, where $p(i)$ is the k -th closest item.
3. Compare candidate distance with boundary plus margin ϵ .
4. Convert the comparison into a binary membership value.

Part	Meaning
$N_{k+\epsilon}(i)$	expanded neighborhood around anchor i
$\mathbb{1}_{N_{k+\epsilon}}(i, j)$	one if j is inside that neighborhood, zero otherwise
$D(i, j)$	squared Euclidean distance from anchor i to candidate j
$p(i)$	index of the item currently at neighbor rank k
$D(i, p(i))$	distance to the current neighborhood boundary
ϵ	nonnegative margin that expands the boundary
$\text{sg}(\cdot)$	stop-gradient operation; use the value but do not differentiate through it
$\theta(\cdot)$	binary threshold function

7.4.2 Why the minus signs produce a distance comparison

The Heaviside function returns one when its input is nonnegative:

$$-D(i, j) + D(i, p(i)) + \epsilon \geq 0.$$

Move $D(i, j)$ to the other side:

$$D(i, j) \leq D(i, p(i)) + \epsilon.$$

Therefore the formula is simply a compact differentiable-in-backward way to ask whether the candidate is no farther than the current boundary plus margin.

7.4.3 Numerical example

Let $k = 4$. Suppose the fourth-smallest distance from anchor i is

$$D(i, p(i)) = 0.70,$$

and let $\epsilon = 0.05$. The effective threshold is

$$0.70 + 0.05 = 0.75.$$

For candidate j with $D(i, j) = 0.72$,

$$-D(i, j) + D(i, p(i)) + \epsilon = -0.72 + 0.70 + 0.05 = 0.03 \geq 0,$$

so the indicator is 1. For candidate r with distance 0.82,

$$-0.82 + 0.70 + 0.05 = -0.07 < 0,$$

so the indicator is 0.

The mature interpretation

Equation P2 converts a continuous distance matrix into a directed binary adjacency matrix. The forward pass constructs an exact threshold graph. The surrogate backward rule lets gradient descent alter distances so that graph edges can appear or disappear.

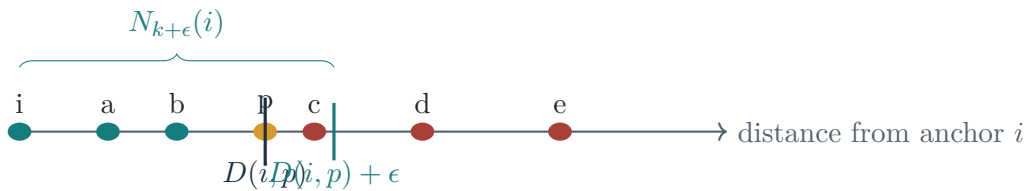


Figure 7.4: ϵ expands the boundary beyond the current k -th neighbor and acts as a ranking margin.

7.5 Why stop the gradient through the boundary?

The operator $\text{sg}(\cdot)$ treats the k -th-neighbor distance as a constant during backpropagation. Otherwise, the optimization could alter the threshold itself instead of moving the candidate pair relative to the threshold. In code, this is the difference between

```
threshold.detach()
```

and an ordinary differentiable threshold.

```
1 D = (2.0 - 2.0 * S).clamp_min(0.0)
2 # topk on -D returns smallest distances
3 neg_dk = (-D).topk(k, dim=1).values[:, -1:]
4 N = GreaterThan.apply(-D + eps, neg_dk.detach())
```

The mask N is binary in the forward pass. Its custom autograd function returns a constant gradient with opposite signs for the two comparison arguments.

7.6 Why not use a sigmoid?

A low-temperature sigmoid approximates a step, but its gradient becomes concentrated in a tiny interval around the boundary. A higher-temperature sigmoid spreads gradients but ceases to behave like a binary neighborhood. The paper reports that the exact-forward, constant-backward construction performs better in its ablations.

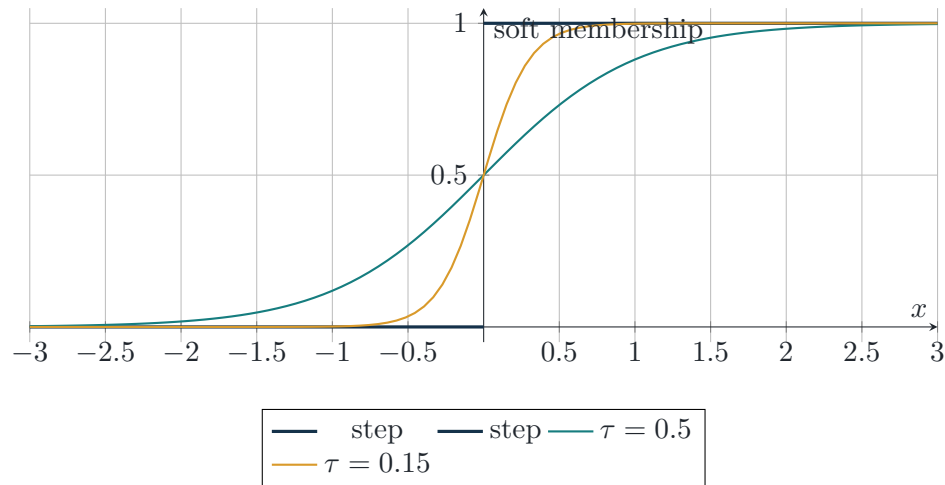


Figure 7.5: A sharper sigmoid is a better forward approximation but has useful gradients over a narrower region.

Step 2: Shared Neighborhood Context

8.1 Start with a social-structure question

Direct similarity asks whether two samples are close. Contextual similarity asks a second question: do the two samples associate with the same surrounding points?

Suppose

$$N(i) = \{i, a, b, c\}, \quad N(j) = \{j, a, b, c\}.$$

The two samples share three surrounding neighbors, so their local contexts are strongly aligned. By contrast, if

$$N(r) = \{r, u, v, w\},$$

then i and r have almost no local context in common.

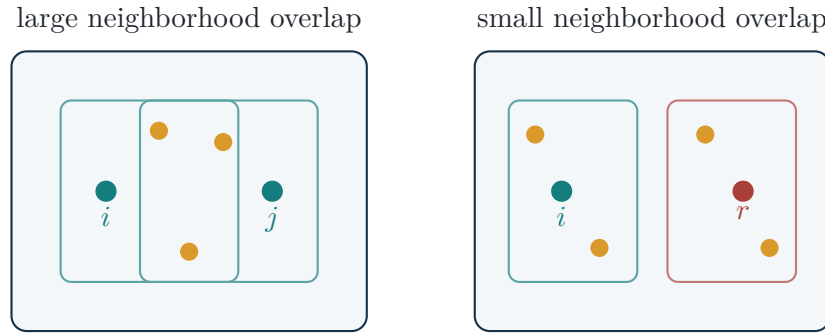


Figure 8.1: Context compares local structures, not only the direct line between two points.

8.2 Turn sets into binary rows

For anchor i , create a row vector whose p -th entry is one when p belongs to i 's neighborhood:

$$N_i = \begin{bmatrix} 1 & 1 & 0 & 1 & 0 \end{bmatrix}.$$

For anchor j , suppose

$$N_j = \begin{bmatrix} 1 & 0 & 0 & 1 & 1 \end{bmatrix}.$$

Multiplying matching coordinates and adding gives

$$N_i N_j^T = (1)(1) + (1)(0) + (0)(0) + (1)(1) + (0)(1) = 2.$$

The result is exactly the number of positions where both rows contain one: the shared-neighbor count.

8.3 Neighbor overlap as a dot product

Let $N_{ij} = \mathbb{1}_{N_{k+\epsilon}}(i, j)$. The i -th row N_i is a binary signature of the neighborhood around i . The number of shared neighbors is

$$M_+(i, j) = \sum_{p=1}^n N_{ip} N_{jp} = \langle N_i, N_j \rangle. \quad (8.1)$$

In matrix form,

$$M_+ = NN^\top. \quad (\text{P22})$$

This is why the method can be implemented compactly with a matrix multiplication.

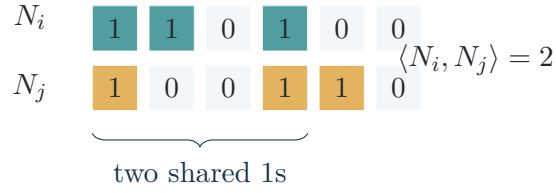


Figure 8.2: Binary neighborhood overlap is an ordinary dot product.

8.4 Why count shared non-neighbors?

Define $\bar{N} = 1 - N$. Then

$$M_-(i, j) = \sum_{p=1}^n (1 - N_{ip})(1 - N_{jp}) = \langle \bar{N}_i, \bar{N}_j \rangle. \quad (8.2)$$

Two neighborhood signatures can be similar because they include the same points *and* exclude the same points. The M_- term also prevents gradients from focusing only on entries where both neighborhood indicators are one.

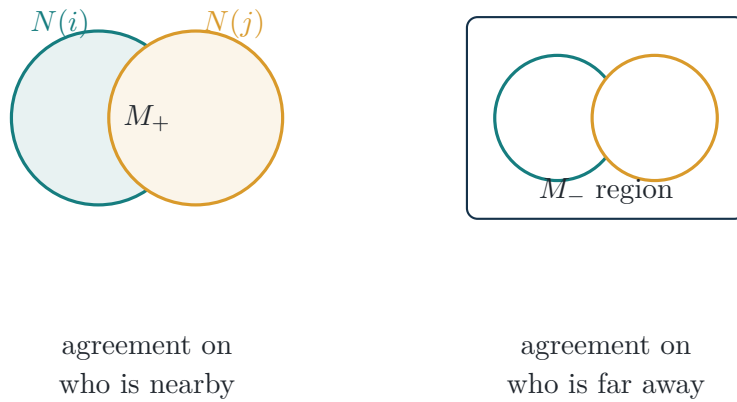


Figure 8.3: M_+ measures shared inclusion; M_- measures shared exclusion.

8.5 The intermediate contextual score W_1

The paper combines the two agreements, normalizes them, and gates the result by whether j is currently a neighbor of i :

$$W_1(i, j) = \frac{N_{ij}}{2} \left(\frac{M_+(i, j)}{\text{sg} \sum_p N_{ip}} + \frac{M_-(i, j)}{\text{sg} \sum_p (1 - N_{ip})} \right). \quad (\text{P3})$$

8.5.1 Read Equation P3 one layer at a time

Rewrite the score as

$$W_1(i, j) = \underbrace{N_{ij}}_{\text{gate}} \times \underbrace{\frac{1}{2}}_{\text{average two agreements}} \times \left(\underbrace{\frac{M_+(i, j)}{\text{sg} \sum_p N_{ip}}}_{\text{shared-neighbor fraction}} + \underbrace{\frac{M_-(i, j)}{\text{sg} \sum_p (1 - N_{ip})}}_{\text{shared-non-neighbor fraction}} \right).$$

Factor	Function
N_{ij}	sets the score to zero unless j is currently a neighbor of i
$M_+(i, j)$	counts samples that are neighbors of both i and j
$\sum_p N_{ip}$	counts the total neighbors of i
$M_-(i, j)$	counts samples that are non-neighbors of both i and j
$\sum_p (1 - N_{ip})$	counts the total non-neighbors of i
$1/2$	averages the inclusion-agreement and exclusion-agreement terms
sg	prevents optimization from changing denominators merely to alter normalization

8.5.2 A complete numerical calculation

Assume a batch of $n = 8$ samples and $k = 4$ neighbors per row. Suppose j is a neighbor of i , so $N_{ij} = 1$. Suppose the two samples share three neighbors and three non-neighbors:

$$M_+(i, j) = 3, \quad M_-(i, j) = 3.$$

Since each row has four neighbors and four non-neighbors,

$$\sum_p N_{ip} = 4, \quad \sum_p (1 - N_{ip}) = 4.$$

Therefore

$$\begin{aligned} W_1(i, j) &= \frac{1}{2} \left(\frac{3}{4} + \frac{3}{4} \right) \\ &= \frac{1}{2} (1.5) \\ &= 0.75. \end{aligned}$$

If $N_{ij} = 0$, the outer gate makes the entire score zero even if the two binary rows agree elsewhere.

Why this is more than pairwise similarity

The value $W_1(i, j)$ depends on many entries of the neighborhood matrix, not only on s_{ij} . Changing one distance can change a neighborhood edge; that edge can change several overlaps; those overlaps can change several contextual scores.

The three design choices are distinct:

1. **Gate N_{ij} :** contextual similarity is initially nonzero only when j belongs to i 's neighborhood.
2. **Average M_+ and M_- :** reward agreement about both local inclusion and exclusion.
3. **Stop-gradient denominators:** normalize the score without training the model to manipulate neighborhood cardinality.

8.6 Algebra when $\epsilon = 0$

When $\epsilon = 0$, every row has exactly k ones, so $\sum_p N_{ip} = k$ and $\sum_p (1 - N_{ip}) = n - k$. The complement overlap simplifies:

$$\begin{aligned} M_-(i, j) &= \sum_p (1 - N_{ip})(1 - N_{jp}) \\ &= n - 2k + \langle N_i, N_j \rangle. \end{aligned} \tag{P12}$$

Substitution gives

$$W_1(i, j) = N_{ij} (a \langle N_i, N_j \rangle + b), \tag{P14}$$

where

$$a = \frac{1}{2k} + \frac{1}{2(n-k)}, \quad b = \frac{n-2k}{2(n-k)}. \tag{8.3}$$

Under the proposition's condition $n > 2k$, both a and b are positive. Thus, if $N_{ij} = 1$, then $W_1(i, j) > 0$ even before considering how large the overlap is.

Graph interpretation. Each row N_i is an adjacency vector in a directed k -nearest-neighbor graph. The dot product $\langle N_i, N_j \rangle$ counts common outgoing neighbors, so W_1 compares the *local graph signatures* of i and j , not merely their direct edge weight s_{ij} .

Step 3: Reciprocal Query Expansion and Symmetry

9.1 Start with two refinements

The paper adds two final refinements to the raw neighborhood-overlap score.

1. **Reciprocity:** trust a close relation more when each point selects the other.
2. **Query expansion:** let a query borrow evidence from its most trustworthy reciprocal neighbors.

9.2 Why nearest-neighbor relations are directional

It is possible for j to be among the nearest neighbors of i while i is not among the nearest neighbors of j . This occurs when local densities differ. A point inside a dense cluster has many very close alternatives; a point in a sparse area does not.

A reciprocal relation requires both directions:

$$j \in N(i) \quad \text{and} \quad i \in N(j).$$

Binary multiplication implements logical AND:

$$1 \cdot 1 = 1,$$

while every other binary product is zero.

9.3 Query expansion as local evidence averaging

Suppose i has two trusted reciprocal neighbors, a and b . Instead of using only the row $W_1(i, :)$, query expansion averages

$$W_1(i, :), \quad W_1(a, :), \quad W_1(b, :).$$

A single unstable relation can therefore be moderated by consistent local evidence.

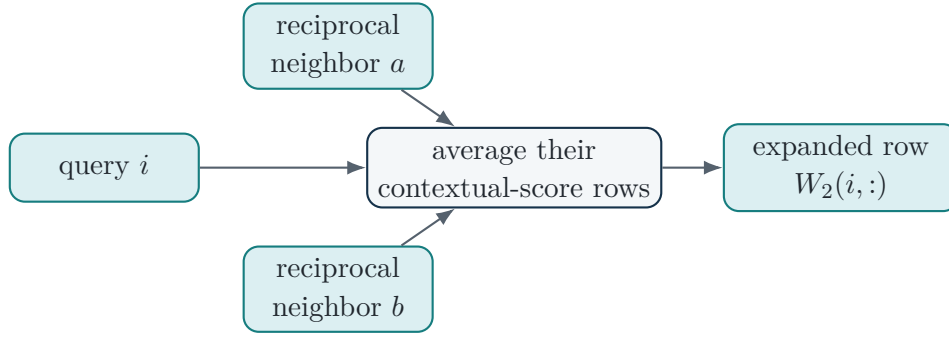


Figure 9.1: Query expansion replaces one point’s estimate with a local consensus estimate.

9.4 Reciprocal close neighbors

The method recomputes a smaller $k/2 + \epsilon$ neighborhood and keeps only reciprocal relationships:

$$R_{k/2+\epsilon}(i, j) = N_{k/2+\epsilon}(i, j)N_{k/2+\epsilon}(j, i). \quad (\text{P4a})$$

A one-sided accidental neighbor is rejected. A mutual neighbor is retained.



Figure 9.2: Reciprocity filters asymmetric nearest-neighbor accidents.

9.5 Query expansion

The method then averages W_1 over the reciprocal neighbors of i :

$$W_2(i, j) = \frac{\sum_p R_{k/2+\epsilon}(i, p)W_1(p, j)}{\sum_p R_{k/2+\epsilon}(i, p)}. \quad (9.1)$$

Finally it symmetrizes:

$$w_{ij} = \frac{1}{2} (W_2(i, j) + W_2(j, i)). \quad (\text{P4})$$

9.5.1 Symbol-by-symbol interpretation of reciprocal query expansion

The reciprocal mask is

$$\mathbb{1}_{R_{k/2+\epsilon}}(i, j) = \mathbb{1}_{N_{k/2+\epsilon}}(i, j)\mathbb{1}_{N_{k/2+\epsilon}}(j, i).$$

Both factors must equal one. The first says j selects i ’s local region; the second says i selects j ’s local region. Their product therefore encodes mutual selection.

The expanded score is

$$W_2(i, j) = \frac{\sum_p \mathbb{1}_{R_{k/2+\epsilon}}(i, p)W_1(p, j)}{\sum_p \mathbb{1}_{R_{k/2+\epsilon}}(i, p)}.$$

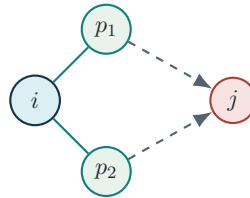
Read the numerator as: add the $W_1(p, j)$ scores over all reciprocal neighbors p of query i . Read the denominator as: count how many reciprocal neighbors were included. Dividing produces an average.

Symbol	Meaning
p	temporary index ranging over possible reciprocal neighbors
$R_{k/2+\epsilon}(i)$	trusted mutual-neighbor set of query i
$W_1(p, j)$	preliminary contextual opinion of neighbor p about candidate j
$W_2(i, j)$	local consensus opinion for query i about candidate j
w_{ij}	final symmetric contextual similarity

Finally,

$$w_{ij} = \frac{W_2(i, j) + W_2(j, i)}{2}$$

averages both directional estimates. This guarantees $w_{ij} = w_{ji}$.



Statistical intuition. Query expansion resembles local smoothing: a noisy pairwise estimate is replaced by an average across a small trusted neighborhood. Averaging can reduce variance, while reciprocity limits contamination from unreliable neighbors.

The Complete Objective and the Role of Every Term

10.1 Begin with the idea of matching two matrices

By this stage, the model has produced a contextual-similarity matrix

$$W = [w_{ij}],$$

and the labels have produced a target matrix

$$Y = [y_{ij}].$$

For a same-label pair, the desired target is $y_{ij} = 1$. For a different-label pair, the desired target is $y_{ij} = 0$. The simplest way to penalize disagreement is squared error:

$$(y_{ij} - w_{ij})^2.$$

y_{ij}	w_{ij}	error	squared error
1	0.9	0.1	0.01
1	0.2	0.8	0.64
0	0.1	-0.1	0.01
0	0.8	-0.8	0.64

Averaging this quantity across all distinct pairs produces the contextual loss.

10.2 Why the final objective contains three terms

The final loss combines three pressures:

- contextual structure should match the label structure;
- direct cosine similarities should remain locally meaningful across batches;
- the whole embedding space should not collapse into a narrow angular region.

No single term performs all three roles equally well.

10.3 Contextual loss

The final contextual matrix $W = [w_{ij}]$ is trained to match the label-similarity matrix Y :

$$\mathcal{L}_{\text{context}} = \frac{1}{n^2} \sum_{i \neq j} (y_{ij} - w_{ij})^2. \quad (\text{P5})$$

10.3.1 Read the contextual loss as four nested operations

The formula

$$\mathcal{L}_{\text{context}} = \frac{1}{n^2} \sum_{i \neq j} (y_{ij} - w_{ij})^2$$

performs four operations:

1. compare target y_{ij} with prediction w_{ij} ;
2. square the error;
3. add errors over all distinct pairs;
4. divide by n^2 to keep the scale approximately stable as batch size changes.

Part	Meaning
$\mathcal{L}_{\text{context}}$	one nonnegative scalar loss for the entire mini-batch
n	number of batch samples
$i \neq j$	exclude trivial self-pairs
y_{ij}	binary target relation
w_{ij}	learned contextual-similarity prediction in $[0, 1]$
$(y_{ij} - w_{ij})^2$	squared disagreement for one pair

The loss equals zero only when every included squared term equals zero. Since a square is zero exactly when its argument is zero, this requires

$$w_{ij} = y_{ij} \quad \text{for every } i \neq j.$$

This observation is central to the ranking proposition.

The diagonal is excluded because every sample is trivially identical to itself.

The derivative with respect to a contextual score is ordinary MSE:

$$\frac{\partial \mathcal{L}_{\text{context}}}{\partial w_{ij}} = \frac{2}{n^2} (w_{ij} - y_{ij}). \quad (10.1)$$

The unusual part is the chain from w_{ij} back through query expansion, neighborhood intersections, and discrete threshold comparisons to the embedding similarities.

10.4 Similarity regularizer

The paper also regulates the mean pairwise cosine similarity:

$$\mathcal{L}_{\text{reg}} = \left(\tilde{s} - \frac{1}{n^2} \sum_{i,j} s_{ij} \right)^2. \quad (\text{P7})$$

If all embeddings crowd into a small angular region, their mean similarity becomes too high. This term encourages broader use of the unit sphere.

10.5 The full objective

$$\boxed{\mathcal{L}_{\text{ours}} = \lambda \mathcal{L}_{\text{context}} + (1 - \lambda) \mathcal{L}_{\text{contrast}} + \gamma \mathcal{L}_{\text{reg}}.} \quad (\text{P6})$$

10.5.1 How the weighting coefficients work

The coefficients λ and γ do not represent probabilities. They determine relative gradient scale.

- $\lambda = 1$ removes the contrastive component and uses contextual loss plus regularization.
- $\lambda = 0$ removes contextual loss and uses contrastive loss plus regularization.
- an intermediate λ combines both signals.
- $\gamma = 0$ disables the mean-similarity regularizer.

The coefficient $(1 - \lambda)$ is a design choice that makes the two principal weights sum to one. It does not imply that the two unweighted losses naturally have equal numerical scale.

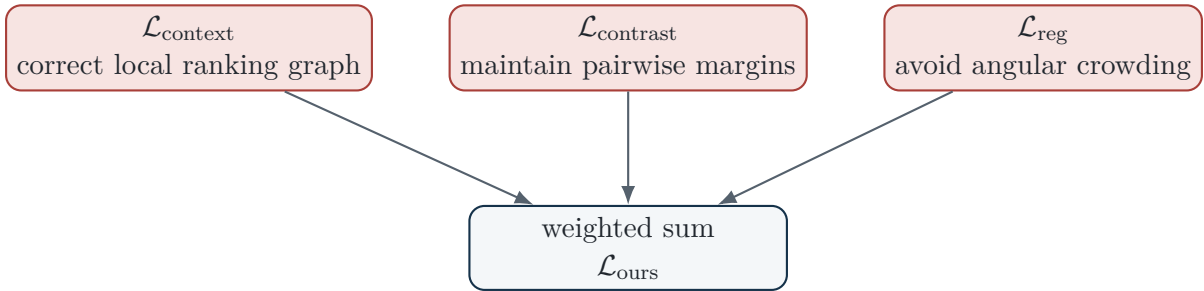


Figure 10.1: The three terms solve different failure modes.

10.6 The decomposability gap

A batch can be perfectly ranked even while the entire dataset is not. Once $\mathcal{L}_{\text{context}} = 0$ for an easy batch, that batch provides no further signal. The pairwise contrastive term continues enforcing absolute margins and helps bridge this difference between batch-wise and dataset-wise ranking.

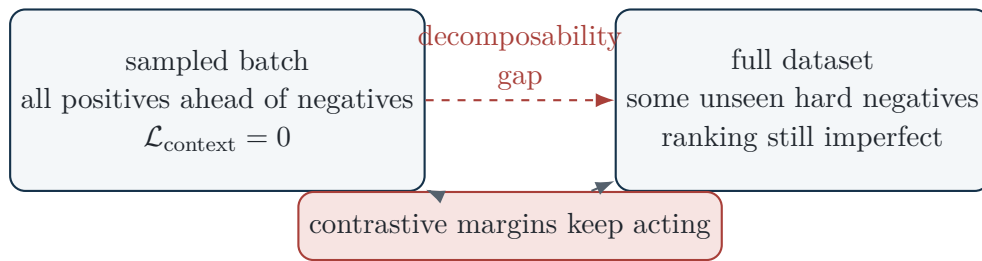


Figure 10.2: Why the hybrid objective is stronger than contextual loss alone.

10.7 Hyperparameter map

Parameter	Mathematical role	Practical interpretation
k	neighbor count	must equal samples per class in the balanced batch under the paper's theory
ϵ	expands the k -th-neighbor threshold	a flexible ranking margin in squared-distance units
α	surrogate derivative of θ	scales gradients through comparisons; largely redundant with learning rate
δ_+	positive similarity margin	positives below it are pulled closer
δ_-	negative similarity margin	negatives above it are pushed apart
λ	contextual-contrastive mixture	1 means contextual only; 0 means contrastive only
γ	similarity-regularizer weight	controls pressure to use more of the sphere
$\tilde{s}$	desired mean cosine similarity	target average angular spread

Why Zero Contextual Loss Means Correct Ranking

11.1 How to read a mathematical proposition

A proposition has assumptions and a conclusion. The assumptions describe the setting in which the statement is guaranteed. Removing an assumption can make the conclusion false.

The phrase “if and only if,” written $\iff$, contains two directions:

$$A \implies B \quad \text{and} \quad B \implies A.$$

For this paper:

- direction 1 asks whether a correct ranking forces contextual loss to zero;
- direction 2 asks whether zero contextual loss forces the ranking to be correct.

The second direction is usually more informative because it rules out an incorrect ranking that somehow achieves a perfect loss value.

11.2 What “correct ranking” means formally

For anchor i , let p be any positive sample with $y_{ip} = 1$, and let j be any negative sample with $y_{ij} = 0$. Correct ranking means

$$s_{ip} > s_{ij}.$$

Read this as: every same-class sample must have larger cosine similarity to the anchor than every different-class sample.

Because distance decreases when similarity increases, the equivalent distance statement is

$$D(i, p) < D(i, j).$$

11.3 The paper’s proposition

Proposition 11.1 (Liao–Tsiligkaridis–Kulis, Proposition 4.1). *Consider a balanced batch of size n with exactly k samples from each class, n divisible by k , $n > 2k$, $k \geq 2$, and $\epsilon = 0$. Then*

$$\mathcal{L}_{\text{context}} = 0$$

if and only if every positive sample is ranked ahead of every negative sample for every anchor:

$$s_{ip} > s_{ij} \quad \text{whenever} \quad y_{ip} = 1, \ y_{ij} = 0.$$

11.3.1 Translate every quantifier in the ranking condition

The paper writes a condition of the form

$$s_{ip} > s_{ij}, \quad \forall p, j \text{ such that } y_{ip} = 1 \text{ and } y_{ij} = 0, \quad \forall i.$$

Read the quantifier $\forall$ as “for every.” The complete sentence is:

For every anchor sample i , for every same-label sample p , and for every different-label sample j , the similarity between i and p must be greater than the similarity between i and j .

This is stronger than requiring the nearest neighbor to be correct. It requires all positives to appear ahead of all negatives inside the batch.

Why counting the anchor itself matters

Each class contributes exactly k samples to the batch, including the anchor. Under perfect ranking, the first k positions are precisely the complete same-class group. This is why the neighborhood size and samples-per-class count must match in the proposition.

Plain-language translation

Under the balanced-batch assumptions, the contextual loss has the right global minimum: it is zero exactly when every anchor’s same-class examples occupy the full top- k neighborhood and every different-class example lies below them.

11.4 Forward direction: correct ranking implies zero loss

Assume every sample is correctly ranked.

Step 1. Each class contributes exactly k examples. Therefore the top- k neighborhood of anchor i is exactly its class:

$$N_{ij} = y_{ij}.$$

Step 2. If $N_{ij} = 1$, then i and j have identical neighborhoods of size k , so

$$\langle N_i, N_j \rangle = k.$$

Step 3. Substituting into the simplified W_1 formula yields $W_1(i, j) = N_{ij} = y_{ij}$.

Step 4. Reciprocal query expansion averages identical rows inside the same class, so it leaves these values unchanged.

Step 5. Hence $w_{ij} = y_{ij}$ for all off-diagonal pairs and the MSE is zero.

11.5 Backward direction: zero loss implies correct ranking

Assume $\mathcal{L}_{\text{context}} = 0$ but ranking is not correct. Because the batch contains exactly k positives for each anchor, including itself, a missing positive from the top- k set forces at least one negative intruder into

the top- k set. So there exists a negative pair (i, j) with

$$y_{ij} = 0, \quad N_{ij} = 1.$$

From equation (P14), $a > 0$ and $b > 0$, hence

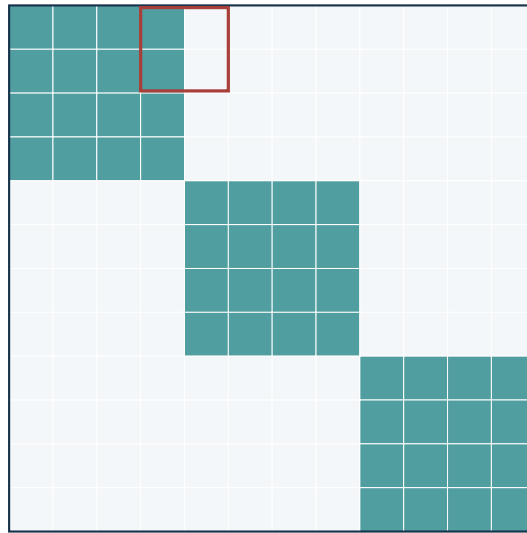
$$W_1(i, j) = a \langle N_i, N_j \rangle + b > 0.$$

Query expansion is an average of nonnegative quantities with positive weights, so the corresponding contextual value remains positive:

$$w_{ij} > 0.$$

But $y_{ij} = 0$, so $(y_{ij} - w_{ij})^2 > 0$, contradicting zero loss. Therefore ranking must be correct.

perfect-neighborhood matrix $N = Y$



a negative intruder creates
a nonzero off-block entry

Figure 11.1: At the optimum, the neighbor graph has exactly the same block structure as label agreement.

11.6 What the assumptions are doing

Assumption	Why it matters
balanced k samples per class	makes the desired top- k set exactly equal to the same-label set
$\epsilon = 0$	fixes each neighborhood cardinality at exactly k
$n > 2k$	ensures the constant $b = (n - 2k)/(2(n - k))$ is positive in the proof
self included	makes a class of k examples correspond to a neighborhood of size k
outside these assumptions	the loss may remain useful empirically, but the proposition's zero-loss/correct-ranking equivalence is not guaranteed

Gradient Intuition and Semantic Consistency

12.1 Begin with the difference between a forward value and a backward signal

The forward pass answers: what score and loss do the current embeddings produce? The backward pass answers: in which direction should the embeddings and parameters change?

These are separate questions. A function may produce a sensible forward value but have a derivative that is unusable for learning. The hard indicator is the main example: its forward output correctly represents membership, but its true gradient is zero almost everywhere.

12.2 What a pairwise gradient means geometrically

Consider a similarity s_{ij} . If

$$\frac{\partial \mathcal{L}}{\partial s_{ij}} < 0,$$

then increasing s_{ij} would decrease the loss, so gradient descent tends to make the pair more similar. If

$$\frac{\partial \mathcal{L}}{\partial s_{ij}} > 0,$$

then decreasing s_{ij} would decrease the loss, so gradient descent tends to separate the pair.

For contextual loss, this derivative is influenced by many neighborhood relations. A pair can receive a gradient not only because of its own label but because changing that pair changes neighborhood membership and therefore changes contextual scores for several other pairs.

12.3 Simplified gradient view

For $\epsilon = 0$, the paper writes

$$W_1(i, j) = N_{ij} (a \langle N_i, N_j \rangle + b). \quad (\text{P8})$$

Consider a negative intruder $j \in N(i)$, so $y_{ij} = 0$ but $N_{ij} = 1$. The MSE wants to reduce $W_1(i, j)$. It can do so in two ways:

1. remove the direct neighborhood edge $i \rightarrow j$;
2. reduce overlap between the neighborhood signatures N_i and N_j .

The second route changes distances between i and members of j 's neighborhood. That is the key distinction from a purely pairwise loss.

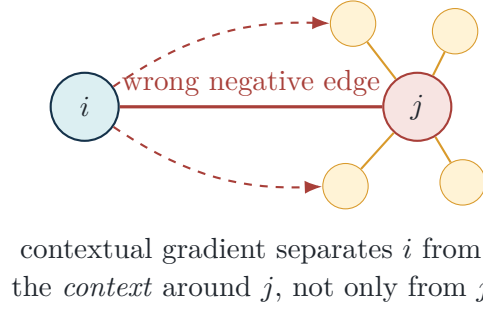


Figure 12.1: A contextual error propagates through a neighborhood.

12.4 Why apparently “wrong” pairwise gradients can generalize

The paper observes that a contextual gradient may sometimes pull apart same-label examples or push together different-label examples. Pairwise supervision would call that wrong. Contextual supervision asks a different question: is the binary label relation consistent with the local semantic structure?

The paper’s gradient-clamping experiment enforces the pairwise label signs

$$\partial D(i, j) \leftarrow \begin{cases} \max(\partial D(i, j), 0), & y_{ij} = 1, \\ \min(\partial D(i, j), 0), & y_{ij} = 0, \end{cases} \quad (\text{P10})$$

and reports higher train retrieval but lower test retrieval. Its interpretation is that some label-inconsistent gradients act as regularization against semantically inconsistent labels.

This argument concerns noisy or semantically incomplete *labels*. Pose-estimation corruption is noisy *input*. A corrupted skeleton may move an entire local neighborhood coherently or destroy it. Therefore the claim “context helps corrupted poses” remains an empirical hypothesis.

12.5 Stepwise gradient interpretation

The supplementary analysis can be summarized as follows:

Component	Positive pair error	Negative pair error
Step 1: N_{ij}	positive outside top- k is pulled inward	negative inside top- k is pushed outward
Step 2: overlap	align the two local contexts	separate each sample from the other’s local context
Step 3: query expansion	smooth toward trusted-neighbor consensus	suppress one-sided accidental relations

A Worked Numerical Example

13.1 Setup

Use a balanced batch with $n = 12$, three classes, and $k = 4$ samples per class:

$$A = \{1, 2, 3, 4\}, \quad B = \{5, 6, 7, 8\}, \quad C = \{9, 10, 11, 12\}.$$

Suppose anchor 1 has a damaged neighborhood

$$N(1) = \{1, 2, 3, 5\},$$

so positive 4 is missing and negative 5 is an intruder. Let

$$N(2) = \{2, 1, 3, 4\}, \quad N(5) = \{5, 6, 7, 1\}.$$

13.2 A true positive pair: (1, 2)

The shared neighbors are

$$N(1) \cap N(2) = \{1, 2, 3\}, \quad M_+(1, 2) = 3.$$

The union contains five items, so seven of the twelve items are excluded by both neighborhoods:

$$M_-(1, 2) = 12 - |N(1) \cup N(2)| = 12 - 5 = 7.$$

Because $2 \in N(1)$, the gate is one. Therefore

$$W_1(1, 2) = \frac{1}{2} \left(\frac{3}{4} + \frac{7}{8} \right) \tag{13.1}$$

$$= \frac{13}{16} = 0.8125. \tag{13.2}$$

The target is $y_{12} = 1$, so this pair contributes

$$(1 - 0.8125)^2 \approx 0.0352$$

before the global $1/n^2$ factor.

13.3 A wrong negative pair: (1, 5)

Here

$$N(1) \cap N(5) = \{1, 5\}, \quad M_+(1, 5) = 2.$$

The union has six items, so $M_-(1, 5) = 6$. Since $5 \in N(1)$,

$$W_1(1, 5) = \frac{1}{2} \left(\frac{2}{4} + \frac{6}{8} \right) \quad (13.3)$$

$$= 0.625. \quad (13.4)$$

But $y_{15} = 0$, so the contribution is

$$(0 - 0.625)^2 = 0.390625.$$

This is a strong signal to remove the intruder and separate the surrounding contexts.

13.4 A missing positive pair: (1, 4)

Because $4 \notin N(1)$, the gate makes

$$W_1(1, 4) = 0$$

even though $y_{14} = 1$. Its squared error is 1, the largest possible. Step 1 therefore pulls sample 4 toward the top- k boundary of anchor 1.

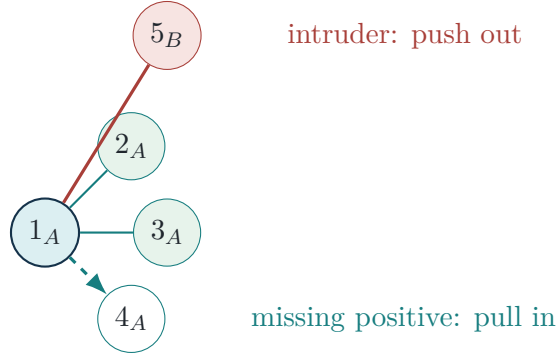


Figure 13.1: The numerical example as a neighborhood-repair problem.

13.5 What query expansion would do

Suppose 1 and 2 are reciprocal close neighbors. Then $W_2(1, j)$ averages the row of W_1 for anchors 1 and 2. Since anchor 2 has the correct neighbor 4 and excludes intruder 5, its row supplies a corrective vote: $W_2(1, 4)$ rises and $W_2(1, 5)$ falls. This is the local-consensus interpretation of query expansion.

Adapting the Mathematics to Pose Sequences

14.1 The input tensor

A pose sequence can be written

$$x \in \mathbb{R}^{T \times J \times C}, \quad (14.1)$$

where T is frames, J is joints, and $C \in \{2, 3\}$ is coordinate dimension. A sequence encoder produces one vector:

$$h = f_\theta(x, m), \quad z = \frac{g_\phi(h)}{\|g_\phi(h)\|_2}. \quad (14.2)$$

Here m can be a validity mask for missing joints or frames, f_θ is MotionBERT or another temporal encoder, and g_ϕ is a small projection head.

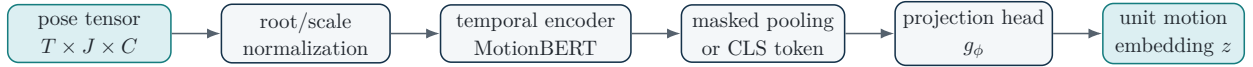


Figure 14.1: The loss does not care whether z came from an image or a motion encoder. Only the encoder-side computation changes.

14.2 Pose normalization

For joint j at frame t , let r be a root joint and b a body-scale statistic such as mean bone length. A simple normalization is

$$\tilde{x}_{t,j} = \frac{x_{t,j} - x_{t,r}}{b + \eta}, \quad (14.3)$$

with small $\eta > 0$ for stability. This removes global translation and much of body-size variation. View normalization, if used, should be treated as a separate design choice because it can remove information or create artifacts.

14.3 Controlled corruption operators

The main experiment corrupts the query and leaves the gallery clean. Let corruption severity be c .

14.3.1 Gaussian joint jitter

$$\tilde{x}_{t,j} = x_{t,j} + \sigma(c)\xi_{t,j}, \quad \xi_{t,j} \sim \mathcal{N}(0, I_C). \quad (14.4)$$

Use scale-normalized coordinates so σ has comparable meaning across subjects.

14.3.2 Joint or limb dropout

Let $m_{t,j} \sim \text{Bernoulli}(1 - p(c))$. Then

$$\tilde{x}_{t,j} = m_{t,j}x_{t,j}, \quad (14.5)$$

while a companion mask tells the encoder which zeros are missing observations. Limb dropout uses a correlated mask over an anatomically connected subset.

14.3.3 Frame dropout

With frame mask $u_t \sim \text{Bernoulli}(1 - q(c))$,

$$\tilde{x}_t = u_t x_t, \quad (14.6)$$

or remove the frame and resample the remaining sequence to length T .

14.3.4 View rotation in 3D

For a yaw angle $\psi(c)$,

$$\tilde{x}_{t,j} = R_y(\psi)x_{t,j}, \quad R_y(\psi) = \begin{bmatrix} \cos \psi & 0 & \sin \psi \\ 0 & 1 & 0 \\ -\sin \psi & 0 & \cos \psi \end{bmatrix}. \quad (14.7)$$

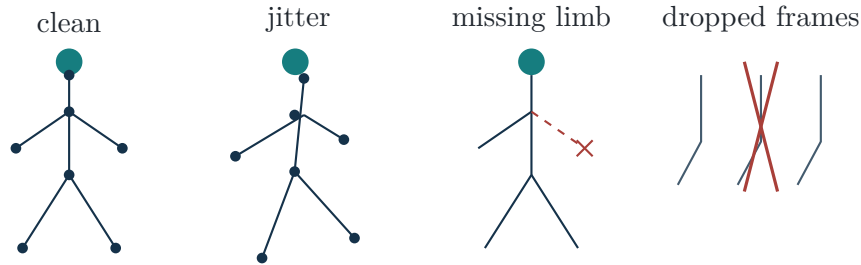


Figure 14.2: Schematic pose corruptions. These are controlled test interventions, not claims that one synthetic operator fully represents a real pose estimator.

14.4 The central hypothesis in mathematical form

Let $M_\ell(c)$ be a retrieval metric for loss ℓ at corruption severity c . Define degradation

$$\Delta_\ell(c) = M_\ell(0) - M_\ell(c). \quad (14.8)$$

The proposed hypothesis is not merely that contextual loss has larger clean $M(0)$, but that

$$\Delta_{\text{context}}(c) < \Delta_{\text{baseline}}(c) \quad \text{over a meaningful severity range.} \quad (14.9)$$

Implementation Blueprint

15.1 Recommended staged implementation

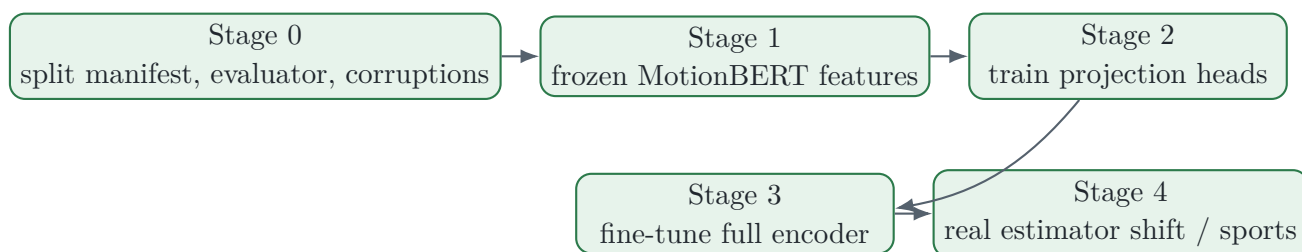


Figure 15.1: A low-risk sequence that isolates bugs and scientific effects.

1. **Freeze the encoder first.** Validate query/gallery construction, cosine ranking, metrics, and deterministic corruption curves.
2. **Train only a projection head.** Compare losses while holding the representation backbone fixed.
3. **Fine-tune the encoder only after stability.** This prevents an encoder bug or optimizer difference from masquerading as a loss effect.
4. **Add a corruption-aware baseline.** Compare ordinary SupCon with and without the same corruption augmentation, then add a MaskCLR-style control if feasible.

15.2 Reference PyTorch implementation

The following implementation follows the paper’s equations, parameterizes k , uses exact forward masks with custom backward gradients, and handles empty contrastive sets safely.

```

1 from __future__ import annotations
2
3 from dataclasses import dataclass
4 from typing import Dict, Tuple
5
6 import torch
7 import torch.nn as nn
8 import torch.nn.functional as F
9
10
11 class GreaterThanSTE(torch.autograd.Function):
12     """Exact >= in forward; constant surrogate gradient in backward."""
13
14     @staticmethod
15     def forward(ctx, x: torch.Tensor, y: torch.Tensor, alpha: float):

```

```

16     ctx.alpha = float(alpha)
17     return (x >= y).to(dtype=x.dtype)
18
19     @staticmethod
20     def backward(ctx, grad_output: torch.Tensor):
21         a = ctx.alpha
22         return grad_output * a, -grad_output * a, None
23
24
25 def contextual_similarity(
26     embeddings: torch.Tensor,
27     *,
28     k: int,
29     eps: float,
30     alpha: float = 10.0,
31 ) -> Tuple[torch.Tensor, Dict[str, torch.Tensor]]:
32     """Compute W from normalized or unnormalized [n, d] embeddings."""
33     if embeddings.ndim != 2:
34         raise ValueError("embeddings must have shape [batch, dim]")
35     n = embeddings.shape[0]
36     if k < 2 or k > n:
37         raise ValueError(f"k must be in [2, {n}], got {k}")
38     if k % 2 != 0:
39         raise ValueError("k must be even for the k/2 reciprocal step")
40
41     z = F.normalize(embeddings, p=2, dim=1)
42     S = z @ z.T
43     D = (2.0 - 2.0 * S).clamp_min(0.0)
44
45     # Step 1: k+eps neighborhood.
46     neg_topk = (-D).topk(k, dim=1).values
47     neg_dk = neg_topk[:, -1:]
48     N = GreaterThanSTE.apply(-D + eps, neg_dk.detach(), alpha)
49
50     # Step 2: shared neighbors and shared non-neighbors.
51     N_count = N.sum(dim=1, keepdim=True).detach().clamp_min(1.0)
52     M_plus = (N @ N.T) / N_count
53
54     N_bar = 1.0 - N
55     N_bar_count = N_bar.sum(dim=1, keepdim=True).detach().clamp_min(1.0)
56     M_minus = (N_bar @ N_bar.T) / N_bar_count
57
58     W1 = 0.5 * (M_plus + M_minus) * N
59
60     # Step 3: reciprocal k/2 neighborhood and query expansion.
61     k2 = k // 2
62     neg_dk2 = neg_topk[:, k2 - 1 : k2]
63     N2 = GreaterThanSTE.apply(-D + eps, neg_dk2.detach(), alpha)
64     R = N2 * N2.T
65     R_count = R.sum(dim=1, keepdim=True).clamp_min(1.0)
66     W2 = (R @ W1) / R_count
67     W = 0.5 * (W2 + W2.T)
68
69     aux = {"z": z, "S": S, "D": D, "N": N, "W1": W1, "R": R}
70     return W, aux
71
72
73 @dataclass(frozen=True)

```

```

74 class ContextualLossConfig:
75     k: int
76     eps: float = 0.05
77     alpha: float = 10.0
78     lam: float = 0.2
79     gamma: float = 0.1
80     target_mean_similarity: float = 0.0
81     positive_margin: float = 0.9
82     negative_margin: float = 0.6
83
84
85 class ContextualMetricLoss(nn.Module):
86     def __init__(self, cfg: ContextualLossConfig):
87         super().__init__()
88         self.cfg = cfg
89
90     @staticmethod
91     def _mean_active(x: torch.Tensor) -> torch.Tensor:
92         active = x > 0
93         if not torch.any(active):
94             return x.new_zeros(())
95         return x[active].mean()
96
97     def forward(self, embeddings: torch.Tensor, labels: torch.Tensor):
98         W, aux = contextual_similarity(
99             embeddings,
100             k=self.cfg.k,
101             eps=self.cfg.eps,
102             alpha=self.cfg.alpha,
103         )
104         S = aux["S"]
105         Y = labels[:, None].eq(labels[None, :]).to(S.dtype)
106         off_diag = 1.0 - torch.eye(S.shape[0], device=S.device, dtype=S.dtype)
107
108         L_context = ((W - Y) ** 2) * off_diag.mean()
109
110         pos_hinge = F.relu(self.cfg.positive_margin - S) * Y * off_diag
111         neg_hinge = F.relu(S - self.cfg.negative_margin) * (1.0 - Y)
112         L_contrast = self._mean_active(pos_hinge) + self._mean_active(neg_hinge)
113
114         L_reg = (S.mean() - self.cfg.target_mean_similarity).square()
115         total = (
116             self.cfg.lam * L_context
117             + (1.0 - self.cfg.lam) * L_contrast
118             + self.cfg.gamma * L_reg
119         )
120         stats = {
121             "loss": total.detach(),
122             "context": L_context.detach(),
123             "contrast": L_contrast.detach(),
124             "reg": L_reg.detach(),
125             "mean_similarity": S.mean().detach(),
126         }
127         return total, stats

```

15.3 Training loop for pose sequences

```

1 for pose, valid_mask, label in train_loader:
2     pose = pose.to(device) # [n, T, J, C]
3     valid_mask = valid_mask.to(device)
4     label = label.to(device)
5
6     h = motion_encoder(pose, valid_mask=valid_mask)
7     z = projection_head(h)
8
9     loss, stats = metric_loss(z, label)
10
11     optimizer.zero_grad(set_to_none=True)
12     loss.backward()
13     torch.nn.utils.clip_grad_norm_(parameters, max_norm=5.0)
14     optimizer.step()

```

15.4 Sampler invariant

Every training batch should satisfy

$$\#\{i : y_i = c\} = k \quad \text{for every class } c \text{ represented in the batch.} \quad (15.1)$$

Use assertions during development:

```

1 unique, counts = torch.unique(labels, return_counts=True)
2 assert torch.all(counts == k), (unique, counts)
3 assert batch_size % k == 0
4 assert k % 2 == 0

```

15.5 Complexity and memory

The dominant contextual operations multiply $n \times n$ matrices, so arithmetic scales as $O(n^3)$ with batch size. The paper also describes empirical time and space scaling as cubic, while emphasizing that GPU matrix multiplication makes the observed cost practical at its studied batch sizes. For pose models, the encoder may dominate compute, but long sequences can make batch size the binding constraint. Profile rather than assume.

Practical levers:

- start with $16 \times 4 = 64$ or $32 \times 4 = 128$ batches;
- shorten or subsample sequences before shrinking k below four;
- use mixed precision for the encoder, but inspect neighborhood ties and numerical stability;
- gradient accumulation does *not* enlarge the contextual neighborhood, because each micro-batch forms a separate graph;
- parameterize k explicitly—the public 256-pixel reference path hard-codes $k = 4$.

Gradient accumulation increases effective optimizer batch size but does not reproduce a larger contextual batch. The loss only sees the current micro-batch's $n \times n$ graph. If neighborhood diversity matters, you need an actual larger forward batch, a memory-bank extension, or a changed method.

15.6 Unit tests before training

1. **Symmetry:** verify $W \approx W^\top$.
2. **Range:** verify $0 \leq W_{ij} \leq 1$ up to floating error.
3. **Perfect blocks:** construct embeddings with well-separated class clusters and check near-zero contextual loss.
4. **Broken pair:** move one negative into an anchor's top- k set and check the loss increases.
5. **Gradient existence:** call backward and verify finite, nonzero gradients.
6. **Sampler:** assert exact k counts per represented class.
7. **Deterministic corruptions:** fixed seed and sample ID must reproduce identical corrupted queries.

Retrieval Evaluation and a Fair Experimental Design

16.1 Begin with one ranked list

Suppose a query has three relevant gallery items. The system returns the following top six labels:

$$[\checkmark, \times, \checkmark, \times, \times, \checkmark].$$

Recall@1 asks whether the first result is relevant. Recall@5 asks whether at least one relevant item appears among the first five. Average precision rewards a ranking that places all relevant items early, not merely one relevant item.

Training loss and evaluation metric are different

The model is optimized using a differentiable training objective. It is judged using retrieval metrics computed from complete ranked lists. A useful training loss is a surrogate: minimizing it should improve the non-differentiable evaluation metrics.

16.2 Query/gallery construction

Use disjoint query, gallery, and validation partitions. The query clip must not occur in the gallery. The primary deployment model is

$$\text{corrupted query} \longrightarrow \text{fixed clean gallery}.$$

A secondary experiment may corrupt both sides, but it answers a different question.

16.3 Recall at K

For query q , let $\pi_q(r)$ be the gallery item at rank r . Then

$$\text{R@K}(q) = \mathbb{1} [\exists r \leq K : \text{rel}(q, \pi_q(r)) = 1]. \quad (16.1)$$

Dataset R@K is the mean over queries.

16.4 Average precision and mAP

Let R_q be the number of relevant gallery items for q , and let $\text{Prec}_q(r)$ be precision among the first r retrieved items. Then

$$\text{AP}(q) = \frac{1}{R_q} \sum_{r=1}^m \text{Prec}_q(r) \text{rel}(q, \pi_q(r)), \quad (16.2)$$

with

$$\text{mAP} = \frac{1}{|\mathcal{Q}|} \sum_{q \in \mathcal{Q}} \text{AP}(q). \quad (16.3)$$

R@1 asks whether the first answer is useful; mAP evaluates the ordering of all relevant examples.

16.5 Robustness curves

Report the raw metric and the clean-to-corrupted drop. A useful summary beyond the proposal is the normalized retention

$$\rho_\ell(c) = \frac{M_\ell(c)}{M_\ell(0) + \eta}. \quad (16.4)$$

This separates clean strength from robustness. It should supplement, not replace, absolute metrics.

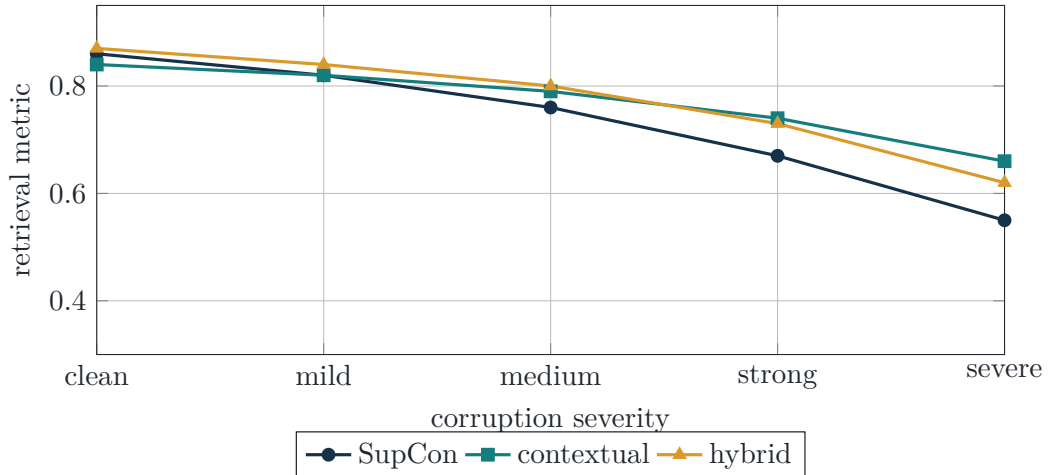


Figure 16.1: Schematic robustness curves only—not experimental results. The target result is a smaller degradation slope, not necessarily the highest clean point.

16.6 Core comparison matrix

Method	Same encoder?	Same corruption exposure?	Scientific role
cross-entropy embedding	yes	matched	classification baseline
triplet or multi-similarity	yes	matched	conventional metric learning
supervised contrastive	yes	matched	strong batch-contrastive baseline
contextual only	yes	matched	isolate neighborhood objective
hybrid contextual+contrastive	yes	matched	paper’s complete mechanism
corruption-aware SupCon	yes	same augmented inputs	separate loss effect from augmentation
MaskCLR-style control	preferably yes	protocol-matched	robustness-specific comparator

16.7 Fairness controls

Following the metric-learning methodology cautions in Musgrave et al., hold fixed:

- encoder architecture and initialization;
- embedding dimension and normalization;
- data split and query/gallery manifest;
- augmentation and corruption exposure;
- number of updates and tuning budget;
- batch composition, except where a method inherently requires a different sampler;
- model-selection metric and validation set;
- reporting over multiple seeds.

16.8 Recommended experiment order

1. clean frozen-feature retrieval;
2. corruption curves with no training;
3. projection-head SupCon vs contextual vs hybrid;
4. full-encoder fine-tuning;
5. corruption-aware training control;
6. cross-subject and cross-setup replication;

7. optional pose-estimator shift;
8. optional SportsPose or fine-grained sports extension.

Discussion Preparation

17.1 A 60-second explanation

Use this almost verbatim

“I want to test whether contextual metric learning transfers from image retrieval to pose-sequence retrieval. A temporal encoder maps each skeleton sequence to a normalized vector, and cosine similarity ranks a clean gallery. Standard metric losses supervise individual pairs or triplets. The contextual objective also compares their local neighborhoods: it builds a differentiable top- k graph, measures shared and reciprocal neighbors, and trains that graph to match label agreement. The original paper shows robustness to label noise, but that does not imply robustness to corrupted pose coordinates. My experiment isolates that question by holding the encoder, data split, batch composition, and corruption protocol fixed while comparing supervised contrastive, contextual, and hybrid losses on clean and increasingly corrupted queries.”

17.2 The five equations to narrate

1. $D(i, j) = 2 - 2s_{ij}$: normalized cosine geometry.
2. $N_{ij} = \theta(-D_{ij} + \text{sg}(D_{i,k}) + \epsilon)$: differentiable neighborhood membership.
3. W_1 : normalized agreement on shared neighbors and shared non-neighbors.
4. W_2 and W : reciprocal query expansion and symmetrization.
5. $\mathcal{L} = \lambda\mathcal{L}_{\text{context}} + (1 - \lambda)\mathcal{L}_{\text{contrast}} + \gamma\mathcal{L}_{\text{reg}}$: ranking, margins, and space usage.

17.3 Likely questions and precise answers

Question	Answer
What is the novelty?	Not pose retrieval itself and not a new encoder. The novelty is a controlled test of whether a supervised neighborhood-ranking objective improves <i>retrieval robustness to corrupted pose inputs</i> .
Why should context help input noise?	It may reduce reliance on one unstable pairwise comparison by aggregating local neighborhood evidence. That is a hypothesis; coherent neighborhood shifts could also make it fail.

Why NTU120 first?	It provides scale, many action classes, standard subject/setup protocols, and compatibility with a public motion-embedding pipeline. It is an engineering and methodology benchmark, not proof of fine-grained technique understanding.
Why k equals samples per class?	Under balanced sampling, perfect top- k neighborhoods then equal the same-label set. This is the condition behind the ranking proposition.
Why not only use SupCon?	SupCon is the strongest simple baseline. The project asks whether explicit neighborhood overlap adds anything beyond many-positive contrastive learning.
What is the cleanest primary endpoint?	The difference in degradation curves under query corruption, with the gallery fixed and clean, reported using R@1/5/10 and mAP.
What would a null result mean?	Robustness comes mainly from encoder pretraining or corruption exposure rather than the neighborhood objective. That is still a useful separation of mechanisms.
What is the main technical risk?	Batch-level cubic work and the need for enough samples per class. Profile 16×4 and 32×4 batches before full experiments.
Could action-class relevance be too coarse?	Yes. It is appropriate for the first falsifiable benchmark, but it cannot support claims about movement quality, technique, or rehabilitation status.
How will you ensure fairness?	Same backbone, initialization, split, sampler, augmentation, tuning budget, and evaluator; only the objective changes in the core comparison.

17.4 Three decisions to ask the professor

1. Is NTU class-level retrieval an appropriate first scientific target, or should relevance be fine-grained from the outset?
2. Should the first deliverable be a single-backbone loss comparison, or include a full MaskCLR-style robustness baseline immediately?
3. Is the strongest framing “contextual metric learning for temporal data” or “retrieval robustness under pose-estimation noise”?

17.5 Claims to avoid

- “Contextual loss is robust to pose noise.” It has not yet been tested.
- “NTU retrieval measures technique quality.” It measures class relevance under the chosen protocol.
- “The project invents a new motion architecture.” The initial strength is controlled integration, not architecture novelty.
- “A higher clean R@1 proves robustness.” Robustness requires degradation curves or cross-domain evaluation.

- “Gradient accumulation solves the batch-size problem.” It does not create a larger contextual graph.

17.6 Final mental model

One sentence per layer

1. **Geometry:** normalized vectors turn cosine ranking into distance ranking.
2. **Graph:** top- k comparisons turn the similarity matrix into a local directed graph.
3. **Context:** shared neighbors and non-neighbors compare graph signatures.
4. **Consensus:** reciprocal query expansion smooths scores through trusted neighbors.
5. **Learning:** MSE matches the contextual graph to label agreement, while contrastive margins and a mean-similarity regularizer stabilize the embedding.
6. **Research question:** does that graph-based inductive bias preserve motion retrieval when the observed skeleton is damaged?

Notation Reference

Symbol	Meaning
x_i	input item; an image in the foundation paper or pose sequence in the proposal
h_i	unnormalized encoder output
$z_i \in \mathbb{R}^d$	unit-normalized embedding
s_{ij}	cosine similarity $z_i^\top z_j$
$D(i, j)$	squared Euclidean distance $2 - 2s_{ij}$
n	mini-batch size
k	samples per class and contextual neighborhood size
y_{ij}	binary label similarity
θ	Heaviside comparison with custom backward gradient
α	constant surrogate derivative for θ
ϵ	neighborhood margin
$N_{k+\epsilon}(i)$	expanded k -nearest-neighbor set of sample i
M_+	count of shared neighbors
M_-	count of shared non-neighbors
W_1	initial gated contextual similarity
$R_{k/2+\epsilon}(i)$	reciprocal close-neighbor set
W_2	query-expanded contextual similarity
w_{ij}	final symmetric contextual similarity
δ_+, δ_-	positive and negative contrastive margins
$\tilde{s}$	target mean cosine similarity
λ, γ	loss-mixture weights

Detailed Algebra and Gradient Notes

B.1 Complement-intersection simplification

Starting from

$$M_-(i, j) = \sum_p (1 - N_{ip})(1 - N_{jp}),$$

expand:

$$M_-(i, j) = \sum_p (1 - N_{ip} - N_{jp} + N_{ip}N_{jp}) \tag{B.1}$$

$$= n - \sum_p N_{ip} - \sum_p N_{jp} + \sum_p N_{ip}N_{jp} \tag{B.2}$$

$$= n - 2k + \langle N_i, N_j \rangle. \tag{B.3}$$

Substituting into W_1 gives the constants a and b in equation (8.3).

B.2 Step-1 surrogate gradient

For the simplified loss

$$\mathcal{L}_1 = \text{MSE}(y_{ij}, N_{ij}),$$

a negative sample inside $N(i)$ receives a constant push outward, while a positive sample outside $N(i)$ receives a constant pull inward. This resembles contrastive loss with a dynamic boundary: the current k -th-neighbor distance replaces a fixed margin.

B.3 Step-2 context gradient

For a wrongly included negative pair and $\epsilon = 0$, the supplementary derivation shows that optimizing neighborhood overlap changes the distance from i to every sample p depending on whether p lies inside j 's neighborhood. In compact qualitative form,

$$\text{sign}\left(\frac{\partial \mathcal{L}}{\partial D(i, p)}\right) = \begin{cases} -1, & p \in N(j), \\ +1, & p \notin N(j), \end{cases}$$

under the paper's sign convention for the considered negative error. Gradient descent therefore increases distance to j 's context and decreases distance to points outside that context. The exact coefficients depend on k , $n - k$, α , and the upstream MSE gradient.

B.4 Logical AND as multiplication

The reciprocal mask uses

$$R = N_2 \odot N_2^T.$$

For binary inputs, multiplication equals logical AND. On the continuous backward surrogate, multiplication is smooth but has zero derivative when the other factor is zero. The paper’s ablation favors multiplication over a minimum-based alternative.

Implementation Checklist

Before the first real run

1. Freeze and version the query/gallery split manifest.
2. Verify no query ID appears in the gallery.
3. Save one deterministic corrupted copy per severity for debugging.
4. Confirm every batch has exactly k examples per represented class.
5. Log $\mathcal{L}_{context}$, $\mathcal{L}_{contrast}$, $\mathcal{L}_{reg}$, mean similarity, and gradient norm separately.
6. Visualize S , N , W_1 , and W as heatmaps for a small batch.
7. Check clean retrieval before any corruption experiment.
8. Compare exact brute-force cosine retrieval against any FAISS index before using approximate search.
9. Tune each baseline on validation data with a matched budget; never select on test corruption curves.
10. Run at least three seeds once the pipeline is stable.

Two-Minute Detachable Cheat Sheet

Core proposition

Question: Does contextual neighborhood optimization preserve pose-sequence retrieval better than standard metric losses as skeleton observations become corrupted?

Design: same MotionBERT encoder, NTU120 split, batch sampler, augmentations, and evaluator; compare SupCon, contextual, and hybrid; corrupt query only against a fixed clean gallery.

Five equations

$$D = 2 - 2S$$

$$N_{ij} = \theta(-D_{ij} + \text{sg}(D_{i,k}) + \epsilon)$$

$$W_1 = \frac{1}{2} \left(\frac{NN^T}{|N_i|} + \frac{(1-N)(1-N)^T}{|N_i^c|} \right) \odot N$$

$$W = \frac{1}{2} \left(\frac{RW_1}{|R_i|} + \left(\frac{RW_1}{|R_i|} \right)^T \right)$$

$$\mathcal{L} = \lambda \text{MSE}(W, Y) + (1 - \lambda) \mathcal{L}_{\text{contrast}} + \gamma \mathcal{L}_{\text{reg}}$$

What makes the method non-standard

Exact binary top- k comparisons in the forward pass; constant heuristic gradients in the backward pass; neighborhood overlap and reciprocal query expansion; k tied to samples per class.

What remains unknown

The paper's label-noise robustness does not prove pose-input robustness. The project must measure corruption degradation curves and include corruption-aware controls.

References

Bibliography

- [1] C. Liao, T. Tsiligkaridis, and B. Kulis. Supervised Metric Learning to Rank for Retrieval via Contextual Similarity Optimization. In *Proceedings of the 40th International Conference on Machine Learning*, 2023.
- [2] P. Khosla et al. Supervised Contrastive Learning. In *NeurIPS*, 2020.
- [3] K. Musgrave, S. Belongie, and S.-N. Lim. A Metric Learning Reality Check. In *ECCV*, 2020.
- [4] S. Kim, D. Kim, M. Cho, and S. Kwak. Self-Taught Metric Learning without Labels. In *CVPR*, 2022.
- [5] J. Liu et al. NTU RGB+D 120: A Large-Scale Benchmark for 3D Human Activity Understanding. *IEEE TPAMI*, 2020.
- [6] R. Memmesheimer, S. Haring, N. Theisen, and D. Paulus. Skeleton-DML: Deep Metric Learning for Skeleton-Based One-Shot Action Recognition. In *WACV*, 2022.
- [7] W. Zhu et al. MotionBERT: A Unified Perspective on Learning Human Motion Representations. In *ICCV*, 2023.
- [8] M. Abdelfattah, M. Hassan, and A. Alahi. MaskCLR: Attention-Guided Contrastive Learning for Robust Action Representation Learning. In *CVPR*, 2024.
- [9] M. Douze et al. The FAISS Library. arXiv:2401.08281.
- [10] C. Liao. Reference implementation: *metric-learning-using-contextual-similarity*. GitHub repository associated with [1].